claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\agent-a4d7a4e88118bba2c.jsonl`
- SHA-256: `37a655c630ce22bf32dd6b9f5f778cf6d9f30dd3b10b4f653ce0014cd292c1ec`
- Source modified: `2026-06-14T09:54:47+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `subagents`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T09:49:32.070Z)

```
Collect ALL the golden-set numbers for the Gate-A/B foundational decision and write an
implications summary. Owner's framing: NUMBERS INFORM, FOUNDATIONS DECIDE (extensibility, scope
of exploration, rigid foundational choices weigh more than raw numbers) ? but make it a data-
INFLUENCED choice. In the badminton-highlight-indexer repo (cwd
C:/Users/avidu/Projects/badminton-highlight-indexer): READ-ONLY on tracked code; you may RUN
evals and WRITE only to scratch/.

Inputs: the 6 persisted golden feature JSONs (output/*_golden_features.json), the experiment
harness (backend/eval/experiment.py, backend/eval/fusion_compare.py,
backend/eval/fusion_audio.py), backend/eval/eval_stats.py (honest bootstrap CI + paired sign
test), and existing baselines (eval_baselines/*.json, docs/QUALITY_ITERATIONS.md). FIRST inspect
a sample golden JSON + backend/eval/fusion_golden.py to learn which signals exist as feature
COLUMNS (shuttle / person / audio) vs GATES (Gate-A = net_crossings, Gate-B = players_in_court).

RUN a LEAVE-ONE-SIGNAL-OUT ablation on the golden set: full signal set vs leave-one-out for EACH
signal group (Gate-A, Gate-B, person, audio, and individual shuttle features where sensible).
For each, report held-out ?F1 with bootstrap 95% CI + paired sign test (reuse experiment.compare
+ eval_stats) = that signal's MARGINAL CONTRIBUTION (does removing it drop quality,
quantifiably?). Also the Gate-A specifics: F1 + segment-count/over-seg ratio with Gate-A ON vs
OFF (known: ?F1 +0.074, 6/6, p=0.031). If the JSONs lack audio columns, run `python -m
backend.eval.fusion_audio --augment` first.

ASSEMBLE one comprehensive numbers table (these ablations + existing baselines:
heuristic/Gen-0/wrapper clean-vs-multicourt, fusion P3 person, P4 audio, Gate-A). Be HONEST
about CIs spanning 0 at n=6 (the central caveat).

WRITE scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md: (a) the comprehensive numbers table; (b)
"Gate-A default flip ? implications" covering numbers AND extensibility / scope-of-exploration /
foundational choices (does making Gate-A the default detection constrain or enable future
signal-fusion? does it commit a foundational structure that's hard to reverse?); (c) an honest
recommendation framing (numbers + foundations, not numbers alone).

GPU + Gemini API are AUTHORIZED if a served-pipeline / end-to-end Gemini run on the golden
videos would add real decision value (e.g. end-to-end highlight quality under Gate-A on vs off).
BUT the persisted-feature ablation is CPU/fast and decisive for the fusion layer ? assess
whether a served Gemini run is WORTH it; do NOT burn a long/expensive Gemini run unless it
clearly changes the decision ? FLAG it for the owner with the expected cost/time instead. Run
all CPU evals with real commands and put the ACTUAL measured numbers in the doc (no fabrication;
if a run errors, report it).

Return: the numbers table + the key implications + the recommendation framing + a yes/no-with-
reason on whether a served Gemini run is warranted.
```

2. assistant (2026-06-14T09:49:37.227Z)

```
I'll start by reading the session handoff and current state to ramp up, then inspect the golden
JSONs and eval harness. Let me pull the repo first and read the key files in parallel.
```

3. user (2026-06-14T09:49:40.299Z)

```
Already up to date.
---STATUS---
master
4be16e1 docs(personalization): add PERSONALIZATION_PLAN ? hybrid per-user feature + locked owner decisions (#144)
```

4. user (2026-06-14T09:49:40.396Z)

```
output\adarsh_avi_singles_golden_features.json
output\kushagra_singles_golden_features.json
output\largetest_doubles_golden_features.json
output\gbaaddy_golden_features.json
output\testlarge_short_golden_features.json
output\mahadevpura_singles_golden_features.json
```

5. user (2026-06-14T09:49:40.462Z)

```
"""Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.

Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per shuttle
candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**, and
writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs 100% on
CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
measured by the shipped experiment harness; person features enter only as feature columns the
learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies them).

Run (on the GPU box):
    python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json --out-dir output

Each output `output/<name>_golden_features.json` is the replayable feature substrate ? re-run
`fusion_compare` with different variants/thresholds forever without re-detecting on the GPU.
"""
from __future__ import annotations

import argparse
import json
import os
import time
from typing import Any, Dict, List, Optional

from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
from backend.pipeline.detectors import rally_gate
from backend.pipeline.detectors import fusion_features as ff
from backend.eval import distill_local
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.training import label_candidates
from backend.utils.run_telemetry import RunTelemetry

# The 5 fps/resolution-invariant shuttle features distill_local already computes.
SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
```

```
# The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)


def _derive_video_path(spec: Dict[str, Any]) -> str:
    """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
    golden labels path (``X.rallies.csv`` -> ``X.MP4``, the golden-set naming convention)."""
    if spec.get("video_path"):
        return str(spec["video_path"])
    labels = str(spec["labels"])
    if labels.endswith(".rallies.csv"):
        return labels[: -len(".rallies.csv")] + ".MP4"
    raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't *.rallies.csv")


def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
                  frame_skip: int = 3, iou: float = 0.5,
                  log_every_sec: float = 15.0,
                  telemetry: Optional[Any] = None) -> Dict[str, Any]:
    """One golden video -> a feature record (shuttle + person features per candidate window).

    Returns ``{name, fps, frame_width, candidates:[{start,end,shuttle{},person{},label}], gts}``.
    `detector` (anything with ``detections_over_windows``) is injectable for GPU-free tests;
    when None a real `PersonDetector` is built (loads torchvision on first use).
    `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health snapshots
    from the detection progress callback so a hard kill leaves a forensic runlog.
    """
    fps, fw = float(spec["fps"]), float(spec["frame_width"])
    traj = str(spec["trajectory"])
    points = TrackNetRunner.parse_trajectory_csv(traj)
    intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
    gts = load_golden_gts(str(spec["labels"]))
    labels = label_candidates(intervals, gts, iou)
    # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic estimate
    # (parity with the shuttle net-crossing gate) rather than hard-coding an orientation.
    axis, net_px, _span = rally_gate.estimate_play_axis(points)
    video_path = _derive_video_path(spec)

    if detector is None:
        from backend.pipeline.detectors.person_detector import PersonDetector
        detector = PersonDetector({"indexing": {"use_gpu": True}})

    # Single-pass person detection over ALL candidate windows (open the video ONCE, read
    # forward ? no per-window seek). Then compute features per window (pure, no GPU).
    n = len(intervals)
    win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
    print(f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one pass...", flush=True)
    t0 = time.monotonic()
    state = {"last": t0}

    def _progress(done: int, total: int) -> None:
        if telemetry is not None:
            telemetry.snapshot(done, total, phase=spec["name"])
        now = time.monotonic()
        if now - state["last"] >= log_every_sec or done >= total:
            el = now - t0
```

```
            rate = done / el if el > 0 else 0.0
            eta = (total - done) / rate if rate > 0 else 0.0
            pct = 100 * done // total if total else 0
            print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total} ({pct}%) "
                  f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
            state["last"] = now

    per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
                                                  progress_cb=_progress)

    candidates: List[Dict[str, Any]] = []
    for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
        det = per_window[i]
        det_fw = det.get("frame_width") or fw
        det_fh = det.get("frame_height") or 0.0
        # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-point
        # normalisation fusion_features uses (x/width, y/height).
        if axis == "x":
            net_norm = net_px / det_fw if det_fw > 0 else 0.5
        else:
            net_norm = net_px / det_fh if det_fh > 0 else 0.5
        sf, ef = win_frames[i]
        shuttle_in = [p for p in points if sf <= p.frame <= ef]
        person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
                                            court_polygon=None, net=(axis, net_norm))
        candidates.append({
            "start": float(s), "end": float(e),
            "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
            "person": {k: float(v) for k, v in person.items()},
            "label": int(labels[i]),
        })
    return {
        "name": spec["name"], "fps": fps, "frame_width": fw,
        "candidates": candidates,
        "gts": [[float(a), float(b)] for a, b in gts],
    }


def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
        fresh: bool = False, smallest_first: bool = False) -> List[str]:
    with open(manifest_path, encoding="utf-8") as f:
        specs = json.load(f)
    if smallest_first:
        # Process small videos first for quick feedback (the slow full match lands last);
        # the LOVO result is order-independent. Proxy size by the trajectory CSV (one row/frame).
        specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
                                             if os.path.isfile(s.get("trajectory", "")) else 0))
    os.makedirs(out_dir, exist_ok=True)
    # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so keep a tiny
    # rotating runlog of velocity + machine health for the post-mortem. Constructing it must never
    # break extraction, so guard the build and fall back to no telemetry on any error.
    telem: Optional[RunTelemetry]
    try:
        telem = RunTelemetry(os.path.join(out_dir, "run_telemetry.jsonl"), label="fusion_golden")
    except Exception:  # noqa: BLE001 - telemetry wiring must never break the run it observes
        telem = None
```

```
        written: List[str] = []
        for vi, spec in enumerate(specs, 1):
            out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
            # Resumable: skip a video whose JSON already exists (a re-run picks up where a prior
            # run left off instead of re-detecting). --fresh forces a redo.
            if not fresh and os.path.isfile(out):
                print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted "
                      f"-> {out} (use --fresh to redo)", flush=True)
                written.append(out)
                continue
            print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...", flush=True)
            t0 = time.monotonic()
            if telem is not None:
                telem.event("video_start", spec["name"])
            try:
                data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
            except Exception as e:  # noqa: BLE001 - record machine state at death, then re-raise
                if telem is not None:
                    telem.event("error", spec["name"], err=str(e))
                raise
            n_pos = sum(c["label"] for c in data["candidates"])
            with open(out, "w", encoding="utf-8") as f:
                json.dump(data, f, indent=2)
            if telem is not None:
                telem.event("video_done", spec["name"],
                            windows=len(data["candidates"]), positives=n_pos)
            print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in {time.monotonic() - t0:.0f}s "
                  f"| {len(data['candidates'])} windows ({n_pos} positive) | {len(data['gts'])} golden rallies "
                  f"-> {out}", flush=True)
            written.append(out)
        return written


def main() -> None:
    ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features (fusion P3 GPU step).")
    ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
    ap.add_argument("--out-dir", default="output")
    ap.add_argument("--frame-skip", type=int, default=3)
    ap.add_argument("--iou", type=float, default=0.5)
    ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON already exists")
    ap.add_argument("--smallest-first", action="store_true",
                    help="process smaller videos first for quick feedback (LOVO is order-independent)")
    args = ap.parse_args()
    run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
        smallest_first=args.smallest_first)


if __name__ == "__main__":
    main()
```

6. user (2026-06-14T09:49:40.742Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-452 of 1835 total (86106 tokens, cap 25000). Call Read with offset=453 limit=452 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-

reminder>

1 ---
2 name: current-state
3 description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not only on request). What just landed, what's next, open decisions."
4 metadata:
5 node_type: memory
6 type: project
7 originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
8 ---
9
10 # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12 **Checkpoint:** 2026-06-14 (FUSION P3/P4 MEASURED + extraction COMPLETE + telemetry #134 merged + Cooler Boost). **THE NUMBERS
13 (honest LOVO n=6, IoU?0.5, C1 bootstrap-CI + sign-test):** shuttle-only F1=**0.307**; **+person (P3) ?F1=?0.021** (?6.7%, CI
14 [?0.165,+0.162], sign 2W/4L p=0.688, **CI-clears-0: NO**) ? NO lift, stays default-OFF (no court polygons ? `players_in_court`
15 counts spectators; per-video wash: testlarge +0.40 / gbaaddy ?0.22). **+person+audio (P4 incremental) ?F1=+0.079** (+27.7%, F1
16 0.286?**0.366**, CI [?0.054,+0.203], sign 4W/2L p=0.688, **CI-clears-0: NO**) ? PROMISING (wins 4/6, largetest/mahadevpura
17 +0.27; endpoint 0.366 > shuttle-only 0.307) but n=6 can't confirm. **VERDICT: neither promotable (both CIs span 0); audio is
18 the lead ? need more matches + measure audio on shuttle-only directly (drop the person handicap).** Recorded in
19 `eval_baselines/experiment_leaderboard.json`; honest negative/uncertain results KEPT ([[paper-coauthor-aim]]). **LANDED THIS
20 SESSION:** result logged to `docs/QUALITY_ITERATIONS.md` (#137); `experiment_leaderboard.json` tracked (#139); **weak-signals
21 audit ?** (person/audio/court-bounds all present, default-OFF, no-special-casing, retained ? refuted=false ? [[weak-signals-retained]]);
22 7-page paper-style **quality report PDF** at `OneDrive\Documents\Rally_Detection_Quality_Report.pdf` (gen `scratch/quality_report.py`;
23 conservative academic restyle, strict-reviewer-approved); `FusionGoldenExtract` scheduled task **unregistered** (cleanup); tree
24 clean on `master`. ?? **NEXT (fusion track):** (1) **wire `run_telemetry` (#134) into fusion_golden + the ps1 wrapper** (deferred,
25 ready, mergeable); (2) **measure audio on shuttle-only directly** (quick eval, reuses the 6 `output/*_golden_features.json`, NO new
26 data ? audio's standalone lift without the person handicap); (3) **grow the golden corpus via Roora** (owner's job ? gives power
27 to confirm the +0.079 audio lead) + **activate court polygons** (re-measure person properly ? the ?0.021 is partly a
28 `court_polygon=None` artifact); (4) future **shuttle-dropped** cue ([[weak-signals-retained]]). **OTHER track (per
29 [[session-handoff]] ? the stated IMMEDIATE PICKUP): EXECUTION-POLICY P3 then P4** (owner-gated; #132/#133/#135 done). **Extraction infra** (all working): detached Scheduled Task `FusionGoldenExtract`
30 runs `scratch/extract_until_done.ps1` (self-restart loop + in-process keep-awake + GPU-busy gate: waits if free VRAM<3500MB) ?
31 deaths were `0xC000013A` console-control kills (NOT crash/sleep/TDR/OOM ? verified via event log), auto-recover via 3-min
32 repetition trigger + restart-on-failure. **Cooler Boost** (MSI Creator Center) dropped 85?67°C, SM ~1100?~1300MHz, velocity
33 ~26?~50 fps (the rest is Max-Q power cap, not thermal).
34
35 **Checkpoint:** 2026-06-14 (EXECUTION-POLICY framework: design + P1 + P2 SHIPPED ? **PR #132 (design) + #133 (P1) + #135 (P2)
36 MERGED**, self-merged on green CI). Owner asked to harden the GX010125 Gemini hang as a **policy-driven, per-job,
37 self-scheduling** system (default WAIT_AND_RESUME, ABORT, POLL_EVERY_N_SEC?, **no master node**) with **very high coverage**

(failures cause delays) and **clean poll-logging** (visible but not polluting). **#132** = design doc `docs/EXECUTION_POLICY.md` (validated the owner's architecture: per-job policy = data on the job; self-schedule via re-enqueue; no master ? rides the existing single-worker executor + `jobs` table + crash sweep; DB is the coordinator). **#133 (P1, the actual fix)** = `backend/infrastructure/execution_policy.py`: `ExecutionPolicy` + `run_bounded` (HARD `op_timeout_sec` per-attempt ? the hang-killer; retries failures AND hangs; hung attempt's daemon thread abandoned) + `poll_until` (bounded poll loop). Clocks injected ? **12 tests, 100% module coverage, instant**. Refactored `gemini.py`: `generate_content`?`run_bounded` (a never-returning generate previously blocked FOREVER = the GX010125 hang; now bounded), processing-wait?`poll_until` (**per-poll logs at DEBUG, not an INFO line every 10s** ? the no-pollution ask). 23 gemini tests stay green. **#135 (P2, substrate)** = DB migration **v3** (`jobs.policy` JSON + `next_poll_at`; existing rows survive via ALTER) + `set_job_waiting(delay_sec)` (self-schedule: park 'waiting' until due) + **`claim_due_job()`** (atomic compare-and-set claim of queued/due-waiting ? running; concurrent-safe, no master) + `create_job(policy=)`/`get_job` policy. **Additive, no executor change**; 10 tests; version asserts bumped 2?3. ? **THIS IS THE OWNER'S REAL GPU MACHINE** (torch+CUDA/cv2/WSL/Gemini live). ?? **REMAINING (owner-gated, NOT done ? deliberately not rushed at session tail given the high-coverage bar): P3** = wire the executor (main.py) to a `claim_due_job` self-scheduling loop honoring WAIT_AND_RESUME (release+reschedule) + **resumable AI handoff** (persist per-window confirmation; resume only unconfirmed ? the GX010125 reel becomes exhaustive across resumes, no wasted Gemini spend); **P4** = apply the policy to the WSL/WASB GPU pass + ffmpeg. Each = its own distinct PR. Ties [[paper-coauthor-aim]] indirectly (infra).

Checkpoint: 2026-06-14 (ACTED ON FIRST-A/B FINDINGS: Gate-A PROMOTED + learned FIXED + GX010125 run ? **PR #130 + #131 MERGED** to master `2e5892e?`; self-merged on green CI). ? **THIS IS THE OWNER'S REAL GPU MACHINE** (torch+CUDA, cv2, WSL Ubuntu, Gemini key in .env all present) ? not the dev container the older CLAUDE.md note assumes. **#130 = learned-variant FIX**: additive/default-OFF `Variant.tune_threshold` (operating point on the TRAIN fold) + `calibrate` (Platt/isotonic, C2); validated on n=6 golden ? **mean held-out F1 0.307?0.423 (+0.116), testlarge 0.000?0.300 (un-zeroed), now BEATS CONTROL 0.388**; threshold-in-LOVO is the big lever. **#131 = Gate-A PROMOTED** (first cue promoted via the harness; doc trail in QUALITY_ITERATIONS ? measured win dF1 +0.074, 6/6, p=0.031; recommended `indexing.fusion.min_crossings=2`; **live served-default flip stays OWNER-GATED** until a real fusion-path run confirms ? fusion P2 is unverified on real video) **+ new `docs/PER_VIDEO_OUTPUT_LAYOUT.md`** (one dir per video; fixes the hardcoded `"output"` joins in `trajectory_hybrid.py:~176` so `--output-dir` truly isolates). **GX010125_1080p RUN (offline, no GPU):** ? key finding ? `wasb_runner.run_predict` has **NO cache-hit early-return**, so the full pipeline would RE-RUN WASB (~tens of min on 1080p) + overwrite the cached CSV; the cached trajectory is reusable ONLY via offline re-score. So ran `scratch/run_gx010125_rescore.py` = parse cached `output/wasb/GX010125_1080p__89605e80ed01_wasb.csv` ? 45 candidates ? CONTROL 45 rallies (11.0min) vs **Gate-A 27** (9.3min), Gate-A **dropped 18 low-crossing windows (~1.7min)** of likely held-shuttle/feed FPs ? isolated `output/GX010125_run/` (no pollution; no golden labels for this footage ? divergence not F1). A FULL Gemini-confirmed highlight reel = a separate ~tens-of-min GPU+API run (owner kicks off). Repro: `scratch/{run_gx010125_rescore,ab_n6_lovo,ab_eval_demo}.py`. All in isolated worktrees off master (siblings concurrent). ?? NEXT (owner-gated): real fusion-path run to confirm Gate-A served-side + flip the live default; impl `PER_VIDEO_OUTPUT_LAYOUT` L1; Tier-2/3 §13 corrections; analyzers/reconcilers A1?A5/R1?R3. Ties [[paper-coauthor-aim]].

Checkpoint: 2026-06-14 (FIRST GOLDEN A/B EXPERIMENT + HARNESS HONEST-REPORTING WIRED ? **PR #128 + #129 MERGED** to master `f680b8d`, self-merged on green CI per [[working-agreement-merging]]; owner-directed momentum). Ran the **first real end-to-end A/B on the n=6 golden corpus, 100% OFFLINE** (re-scored persisted WASB trajectories via `distill_local.build_candidates`, no GPU ? the "persist once re-score" thesis proven). One candidate set re-scored three ways: **Gate-A (net_crossings?2) is a MEASURED WIN ? ?F1 +0.074, 6/6 matches, sign-test p=0.031, CI [+0.042,+0.104], seg-count-ratio 1.68?1.01** (over-seg nearly eliminated); the **window-level 5-feature logistic did NOT clear the bar** (?0.081, 2W/4L; testlarge zeroed = threshold/calibration artifact ? C2 motivation; **distinct from the stronger per-frame `rally_seq_proto` 0.583** ? don't conflate). Headline: **the framework held up ? C1 promoted one variant and refused another (right call both ways); C4 surfaced over-seg tIoU hid.** **#128** = archived posterity doc `docs/archives/quality-iterations/2026-06-first-ab-experiment/` (full tables) + archives-index + live QUALITY_ITERATIONS pointer. **#129 = (b) wired C1+C4 into the CORE `experiment.compare()`** (additive/default-ON `honest_stats=True`: `honest` block = per-metric bootstrap CIs + paired ?F1 CI + exact sign test + F1@k + segment_count_ratio; `record_experiment` carries honest ?F1; existing fields byte-unchanged, `honest_stats=False` = legacy shape). Reuses #124 helpers; complements sibling **#127** which wired the same C1 into `fusion_compare` (no dup). 19 existing experiment tests pass + 3 new; ruff+mypy+CI all green. Repro `scratch/ab_n6_lovo.py` + `scratch/ab_eval_demo.py` (machine-side; read gitignored manifest/external golden). All done in **isolated worktrees off master** (siblings merged #121-#127 concurrently). ?? NEXT (owner-gated): promote Gate-A via the no-regression gate (it's a real measured win); fix the logistic (C2 calibration + threshold-inside-LOVO); Tier-2/3 §13 corrections; analyzers/reconcilers impl A1?A5/R1?R3 (none started). Ties [[paper-coauthor-aim]].

Checkpoint: 2026-06-14 (FUSION P3/P4 MEASUREMENT BUILT + GPU EXTRACTION RUNNING ? master `d7b1f51`; suite 514 green). The "modeling" track for the person-cue (P3) + audio (P4) fusion lift ? all default-OFF, no-special-casing, harness-measured. **Merged this run (mine): #121** fusion P3 (`backend/eval/fusion_golden.py` offline person-feature EXTRACTOR + `fusion_compare.py` LOVO driver ? reuses build_candidates/PersonDetector/fusion_features/experiment.compare, NO new scoring); **#122** P4 (`backend/pipeline/detectors/audio_features.py` + `backend/eval/fusion_audio.py` incremental audio cue); **#123** progress-logging + resumability (`--fresh`); **#125** single-pass `PersonDetector.detections_over_windows` (fixed the per-window-seek O(N) pathology ? ~28s/window ? one sequential decode); **#126** `--smallest-first`; **#127** honest ?F1 = bootstrap CI + paired sign test on per-video held-out F1 (wires #124's `eval_stats.paired_delta_ci`; "never a bare mean"). Concurrent sessions merged #119 (analyzers framework), #120 (run.log), #124 (eval-honesty utils: eval_stats/score_calibration/segmentation_metrics). **? GPU EXTRACTION via DETACHED SCHEDULED TASK `FusionGoldenExtract`** (runs `scratch/extract_until_done.ps1` ? Task Scheduler owns it, OUTSIDE Claude's job object ? survives turn/context/session boundaries; in-process keep-awake + self-restart loop until 6/6; clean UTF-8 logs to `output/fusion_golden.log` + `scratch/extract_wrapper.log`).

Loops `python -u -m backend.eval.fusion_golden
105 --manifest output/human_lovo_manifest.json --out-dir output --smallest-first` ? 6
`output/<name>_golden_features.json` (GITIGNORED
106 + RESUMABLE: skips any video whose JSON exists). **3/6 DONE** (testlarge, mahadevpura,
largetest); remaining
107 kushagra?gbaaddy?adarsh(last, 84k frames). **MONITOR:** JSON count + `tail
output/fusion_golden.log` + `nvidia-smi`;
108 `Get-ScheduledTask FusionGoldenExtract|Select State`. **RESTART (resumes, skips done):**
`Start-ScheduledTask -TaskName
109 FusionGoldenExtract`. **STOP:** `Stop-ScheduledTask -TaskName FusionGoldenExtract` +
`Get-Process python|Stop-Process -Force`.
110 ?? **WHY DETACHED (debugged 2026-06-14):** Claude-bg-task runs DIED TWICE silently
(largetest 61%, kushagra 6% @137s), NO Python
111 traceback AND ? verified via Windows event log ? NO sleep (Kernel-Power 42), NO TDR
(Display 4101), NO WER crash, NO RADAR-OOM
112 (no event 1003; 18GB free). Traceless vanish = external `TerminateProcess`; common
factor = both were Claude-managed bg tasks
113 (job-object kill-on-close at a context boundary fits death #1 @ the prior context
summary). Scheduled Task = immune; GPU access
114 CONFIRMED in the interactive task session (53?64% util, 3.1GB). GPU util while running =
20?64% bursty ? batch=1 Faster-RCNN
115 latency-bound (small kernels + CPU NMS + per-frame ByteTrack), NORMAL, model IS on CUDA,
NOT decode-bound; batching the forward
116 pass is the next-extraction speed lever (changes features, so NOT mid-run). ~1.5-2h
total. Stale Claude bg keep-awakes stopped. **DEATH #3 + GPU-job check (2026-06-14):** died a
3RD time (gbaaddy 69%)
117 ? `LastTaskResult=0xC000013A` = STATUS_CONTROL_C_EXIT = terminated by a **console-
control event / CTRL+C** (NOT crash/sleep/TDR/OOM
118 ? explains the no-trace vanish; killed the detached task too, so NOT Claude's job
object). GPU-contention angle CHECKED: **NO
119 competing GPU compute job** (only desktop apps + our 1 python; 5GB free; HW is a
laptop RTX 2070S **Max-Q**, MSI). Mitigations:
120 (1) task re-registered with **3-min repetition trigger + restart-on-failure (count 10,
1-min)** ? auto-recovers within minutes of
121 ANY death; (2) **GPU-busy gate in the ps1** (owner policy = *wait until free, auto-
resume*): before each python launch, if free
122 VRAM <3500MB (another job has the GPU) it waits 30s + re-checks, then launches. **??
WHEN IT COMPLETES ? the remaining steps for
123 the NUMBERS:** (1) `python -m backend.eval.fusion_compare --features-dir output
--record` ? honest P3 LOVO ?F1 (shuttle-only
124 vs +person) w/ CI+sign-test; (2) `python -m backend.eval.fusion_audio --augment
--compare --record` ? P4 incremental audio
125 ?F1; (3) log both to `docs/QUALITY_ITERATIONS.md` I3 (HONEST measured numbers only ?
[[paper-coauthor-aim]]). Honest caveat:
126 NO court polygons ? `players_in_court` counts spectators (3-7 on "single-court"
Mahadevpura); robust signals = motion +
127 `shuttle_player_colocation`. GPU = RTX 2070S, person detection on Windows torch cu124
(NO WSL). Branch protocol held through
128 heavy concurrent merges (#119-127) ? [[gotcha-shared-checkout-branch-collisions]]. On
master, clean.
129 **STORAGE (this session, owner-approved "delete cache + compact WSL"):** deleted the
stale **93GB GX010125 frame cache**
130 (WSL `~/clips` 95?2.3GB; WSL fs used 152?60GB, 896GB free; C: 53?72GB free) ? kept the
2.2GB GX010125 proxy. The WSL
131 `ext4.vhdx` was `--set-sparse true` + `fstrim`'d (946GB trimmed) but this WSL build
(2.3.24) did NOT auto-deallocate on
132 shutdown ? the file is still **~221GB allocated on C:**. **DEFERRED to post-extraction:
the physical ~150GB C: reclaim via
133 `wsl --shutdown` then elevated `diskpart` ? `select vdisk
file="...\CanonicalGroupLimited.Ubuntu_79rhkp1fndgsc\LocalState\ext4.vhdx"`
134 ? `compact vdisk`** (heavy C: I/O would fight the GPU run's video reads). [[gotcha-long-
runs-gpu-wsl]] · [[cache-hygiene-policy]].
135
136 **Checkpoint:** 2026-06-14 (RALLY-DETECTION FRAMEWORK doc + LITERATURE REVIEW + eval-
honesty code ? **PR #119 + #124 MERGED**

to master via self-merge on green CI per [[working-agreement-merging]]; owner-directed). **PR #119** (`docs/ANALYZERS_AND_RECONCILERS.md`): reframed the analyzers/reconcilers design into a **multi-scope signal-fusion framework** (frame detectors / window analyzers / **session analyzers**; warm-up `SessionAnalyzer` + shuttle-state frame-detector folded in) + **§13 = a verified multi-agent literature-review** (8 patterns surveyed, ~40 citations adversarially checked, **ZERO fabrications**; author-line fixes applied). **Verdict (honest):** the skeleton is STANDARD prior art ? candidate-windows+scorer = Temporal Action Localization; reconciler-as-refinement = Temporal Action Segmentation post-proc; "signal not verdict" = late/score-level fusion + stacked generalization (~20 yrs old, already in racket sports). **Only the report-first reconciler-as-linter bundle rated `novel-combination`**; warm-up-as-soft-feature = thinnest gap. **Cite, don't claim**; terminology realigned (producer?labeling function, scorer?stacked meta-learner, LOVO?leave-one-group-out, tIoU?F1@k/mAP). Ties [[paper-coauthor-aim]] (§13 is the honest related-work backbone). **PR #124** implements the **Tier-1 measurement-honesty course corrections C1?C4** as PURE ADDITIVE / default-OFF helpers (no existing compare/LOVO/metrics behaviour changed): `backend/eval/eval_stats.py` (C1 `mean_with_ci`/ `bootstrap_ci`/`paired_sign_test`/`paired_delta_ci` ? never a bare LOVO mean at n?6; C3 `out_of_fold_predictions`/`grouped_folds` ? no stacking leakage), `score_calibration.py` (C2 Platt+isotonic+brier/ECE ? comparable cue confidences), `segmentation_metrics.py` (C4 F1@{10,25,50} + mAP@tIoU + segment_count_ratio ? make over-segmentation tIoU is blind to visible). 24 tests; ruff+mypy clean; all 4 CI lanes green (full-suite coverage floor held). Both PRs done in **isolated git worktrees** (off master) to dodge the recurring shared-checkout branch-flips (siblings merged #117 fusion-P2, #118, #120, #123 concurrently; **#116 closed ? its content had landed via #117**). ?? NEXT (owner-gated): **wire C1?C4 into the DEFAULT harness reporting** (`compare`/`run_regression.py` emit CIs + F1@k/edit-score instead of a bare mean ? changes reported numbers, so its own PR) + the Tier-2/3 corrections in §13 (global reconciler decode, cue-correlation modeling, temporal smoothing, external ShuttleSet eval). Analyzers/reconcilers IMPL (A1?A5/R1?R3) still owner-gated, none started.

Checkpoint: 2026-06-14 (RUN.LOG READABILITY for manual QA ? **PR #120 MERGED ? master `6076eef`**; owner asked ? self-merged on verified-green CI per [[working-agreement-merging]] (squash; remote+local branch deleted); was branch `feat/runlog-readability` off `35a50f3`; CODE, minimal/config-gated). Owner's manual-QA loop = run the tool on a video ? eyeball the rally CSV against `run.log`. Two readability changes: (1) **suppress HTTP chatter** ? new central helper `backend/utils/logging_setup.py::setup_logging(verbose_http=False)` raises `httpx/httpcore/urllib3/google_genai/google.genai/ google.auth/grpc` to WARNING (NOTSET to restore), gated by NEW knob `logging.verbose_http: false` (`LoggingConfig` on `AppConfig`); our `backend.*` loggers untouched; wired into `run_process_and_stitch.py` (at import + re-applied post-config-load). (2) **end-of-run rally summary** ? new pure helper `backend/tools/export.py::format_rally_summary` (next to `segments_to_csv`/`basic_stats`): header (total rallies + total play HH:MM:SS) + table `# start end dur(s) shot_count` from the produced play_segments so log?CSV line up row-for-row; handles metadata dict-or-JSON-string, missing shot_count?`-`; HH:MM:SS via local `_fmt_hms` matching the segmenters' `_fmt_ts`; logged as ONE block at end of `main()` (also on the all-filtered early-return). Tests: `test_logging_setup.py`, `test_rally_summary.py`, +`verbose_http` default/override in `test_config.py`. Suite **477 green**, ruff (`.`) + mypy (`backend training`) clean. ? Commit-msg first attempt mangled (PowerShell `@'?'@` here-string in the *Bash* tool ? literal `@`; amended via `-F file`) ? Bash tool ? PowerShell. **BRANCH-COLLISION CLEANUP (owner asked):** after merge, LOCAL checkout left clean ? on `master`==`origin/master`==`6076eef`, 0 ahead/behind; pruned 6 stale LOCAL branches mapped to MERGED/CLOSED PRs (#92/#104/#111/#106/#102 + #116-closed-redundant) + the stale remote-tracking refs; **kept** `docs/analyzers-framework` (#119 OPEN). Working tree = only the intentional untracked carryovers (`scratch/`, `artifacts/`). REMOTE still has 15 prunable branches (14 merged-PR + #116) ? **offered to prune, owner did NOT answer the question ? left untouched** (must KEEP `feat/khelacharya-shot-intelligence` #11 = deliberately-retained sibling layer per [[sibling-internship-repo]], + #119). ?? NEXT: owner does the machine-side check (run on a real video ? eyeball the quieter log + summary block; dev container can't run the GPU pipeline); remote-branch prune available on request.

Checkpoint: 2026-06-14 (ANALYZERS DOC ? FRAMEWORK + paper framing ? **PR #119 OPEN, owner-gated, do NOT merge**; branch `docs/analyzers-framework` off master `35a50f3`; docs-only). Owner fed two more ideas this session and asked to fold them in and treat the doc as a **reusable framework / research paper**, not a project patch (ties [[paper-coauthor-aim]]). Extended the on-master `docs/ANALYZERS_AND_RECONCILERS.md` (which had landed via #117) from "two seams" into a **multi-scope signal-fusion framework**: producers emit SIGNALS at **3 temporal scopes** ? frame detectors (`RallyCue`) / window analyzers / **session analyzers** ? all consumed as feature columns by the learned scorer (spine unchanged: **no special-casing**, signal ?scorer, never `if/else`). Folded in: (1) **warm-up/practice** as a new **`SessionAnalyzer`** tier (whole-timeline span ? an `in_warmup` membership feature the scorer DOWN-WEIGHTS, NOT a hard `if t<warmup_end: drop`; measurable on golden TODAY since warm-up is already labeled negative); (2) **shuttle-state** (`in_air`/`racket_contact`/`inactive`) placed correctly as a **frame-scope detector** (a MULTI_SIGNAL_FUSION_PLAN `RallyCue`, NOT a window analyzer) ? derive-from-existing-signals first (velocity+visibility+direction-reversals+audio thwack), dedicated per-frame model ONLY where WASB trajectory gaps bite, with the per-frame-label blocker flagged. Added **§10 generality & research positioning** (weakly-supervised temporal segmentation; standard-vs-contributed) with an explicit **attribution honesty caveat**: NO fabricated citations/results, related-work + ablations + >1-sport still owed, n=6 = direction-check only ? the honest backbone the paper aim needs. Diff = `2-seam?framework` (310+/137?). ? Shared checkout had flipped to master mid-task (the sibling's collision lesson recurred) ? re-verified branch, branched fresh, normal commit (no in-flight WIP left to isolate; #117/#118 already merged so all cross-links resolve). PR #116 stays CLOSED (content on master via #117). ?? NEXT (owner-gated): review #119; impl tracks unchanged (fusion P3 LOVO, then analyzers/reconcilers A1?A5 / R1?R3 per the doc's phasing ? each ONE PR, default OFF, promote on measured LOVO lift).

Checkpoint: 2026-06-14 (FUSION P2 SHIPPED + collision cleanup + paper aim ? master `35a50f3`; suite 467 green; 0 open PRs). **PR #117 MERGED = `FusionSegmenter`** (fusion P2, registry `fusion_hybrid`, **default-OFF**): shuttle-primary (windows seed from WASB/TrackNet) + adjudicating cues ? `net_crossings` (Gate A, always persisted, GPU-free via rally_gate w/ a reused axis estimate) + optional person cue (Gate B) persisting the ~5 `fusion_features.py` features (pure-tested) via `PersonDetector.detections_per_frame` over each window's ORIGINAL-video frame range. Plugs in via **two IDENTITY hooks** on `trajectory_hybrid` (`_enrich_candidate_stats`/`_select_for_handoff`) ? wasb/tracknet byte-identical (adversarially verified). Served gating optional+default-OFF (`fusion.min_crossings`/`min_players`) ? gates the handoff while persisting the full SUPERSET (offline substrate intact). Court mask = optional `fusion.court_polygon`. **Design-first** (`fusion-p2-design` workflow) ? **31-agent adversarial review** (25 findings?19 confirmed; headline = NO-REGRESSION verified) ? fixes folded (frame-dim guards, round() frame idx, rally_gate `estimate=` reuse, honest torch docstrings, +6 tests). **PR #118 MERGED** = README index fix (QUALITY_ITERATIONS + EXPERIMENT_HARNESS P1-SHIPPED). **? SHARED-CHECKOUT COLLISION (lesson):** the spawn_task DESIGN session ran in THIS checkout, committed `0f673ff` (the analyzers design doc) + flipped the branch BETWEEN my branch-point check and my `checkout -b` ? my fusion branch rode on top ? **#117's squash absorbed the design doc**, so **`docs/ANALYZERS_AND_RECONCILERS.md` is on master via #117** and **PR #116 was CLOSED as redundant** (content intact on master; #116's stale "OPEN do-not-merge" checkpoint below is superseded). Re-verify `git branch` AND that no sibling commit landed since, before `checkout -b`. **Owner side-aim ratified: co-author a paper** "local, cheap & efficient rally detection via multiple clever ideas" ? [[paper-coauthor-aim]]; `docs/QUALITY_ITERATIONS.md` is the ablation-ready empirical backbone (honest measured results only). **Analyzers+reconcilers DESIGN on master (owner-gated, NOT started):** injectable per-window analyzers (e.g. net-hit service-fault) + a report-first reconciliation pass; spine = signals?learned-scorer, **no special-casing** (owner directive). ?? NEXT (owner-gated): **fusion P3** (person features ? `experiment.compare` LOVO lift on the 6 golden videos ? the eager-person-compute cleanup lands here) + measure the Gate-A held-shuttle drop on GPU; then analyzers/reconcilers impl; P4 audio. Earlier this session: PR #114 harness, #115 person-cue extraction (see checkpoints below).

Checkpoint: 2026-06-14 (DESIGN DOC: two extensibility seams ? **PR #116 OPEN, owner-gated, do NOT merge**; docs-only, branch `docs/analyzers-and-reconcilers` off master `df7d5df`). Produced `docs/ANALYZERS_AND_RECONCILERS.md` (367 lines, a peer to EXPERIMENT_HARNESS + MULTI_SIGNAL_FUSION_PLAN) + a 1-line `docs/README.md` cross-link. **Spine (owner directive): NO special-casing in the decision path** ? every analyzer/reconciler emits a SIGNAL (value+confidence); the SCORING MODEL learns to weight it (already true in miniature: `experiment.predict`/`_gate_mask` treat `net_crossings`/`players_in_court` as feature columns). **Seam 1 = injectable `WindowAnalyzer`s (forward)**: a registry (mirrors `segmenter_registry`) of pure per-window cues over the already-produced per-frame-aligned signals (WASB trajectory + `PersonDetector.detections_per_frame` + `rally_gate` net); each emits an `AnalyzerResult` = confidence-scored features ? `candidate_segments.stats`, events ? `events` table (buffer-on-candidate, flush-on-confirm), optional boundary hints; worked example = a **service-fault** analyzer (shuttle halts in net region ? `service_fault` signal + `net_hit` event + END hint); injects at the FusionSegmenter `_enrich_candidate_stats` hook, flag-gated default OFF. **Seam 2 = reconciliation pass (backward)**: a report-first "linter for rally decisions" ? `ConsistencyRule`s (trailing-dead-time?truncate *(owner's misfire fix)*, no-net-crossing?drop, over-merged?split, boundary-slack?tighten) flag decision-vs-evidence conflicts; **report-first/apply-optional, NEVER silent**; expressible as a harness `Variant` transform so LOVO-measurable on the 6 golden videos BEFORE auto-apply; idempotent, fixed precedence. ? **DESIGNED AGAINST the IN-FLIGHT fusion-P2 WIP** (uncommitted in the shared tree: `fusion_hybrid.py`, `fusion_features.py`, `FusionConfig.cue_flags`, `person_detector.detections_per_frame`, `docs/QUALITY_ITERATIONS.md`) ? that WIP was kept **byte-intact** (I isolated my single README line via git plumbing; tree is CRLF / autocrlf=true, restored owner README to exact session-start bytes 3841B). **Merge-order:** the doc cross-links QUALITY_ITERATIONS.md + the fusion hook, neither on master yet ? **merge #116 after (or with) fusion-P2** (noted in the PR body). ?? NEXT: owner reviews #116; implementation is the phased A1?A3 / R1?R3 plan in the doc (each ONE PR off master, default OFF, promoted only on measured LOVO lift) ? all owner-gated, none started. Ties [[candidate-segments-finetuning]] + the harness ([[working-agreement-merging]]).

Checkpoint: 2026-06-13 (RALLY-QUALITY TRACK: HARNESS + PERSON-CUE EXTRACTION SHIPPED ? **PR #114 + #115 MERGED**, master `df7d5df`; suite 421?441 green; 0 open PRs). **PR #115 (fusion P1):** extracted the torchvision person detector + ByteTrack + velocity windowing out of `yolo_hybrid` into `backend/pipeline/detectors/person_detector.py` (`PersonDetector`, a reusable cue producer); `yolo_hybrid` now ORCHESTRATES only (chunk/prefetch/AI-handoff/DB) and delegates Stage-1 to `self.person`. **No behavior change** ? proven by the unchanged `process_video` output tests (mock seam moved to `self.person`); pure detection unit tests ? new `tests/test_person_detector.py`. person_detector is mypy-CLEAN (`self.model: Any`) so it needed NO quarantine; the torchvision code that put yolo_hybrid in quarantine left it, but yolo_hybrid STAYS quarantined for PRE-EXISTING orchestration debt (implicit-Optional process_video defaults, Success|Failure ABC-override mismatch, a Future[...] annotation ? flagged in mypy.ini to ratchet separately). ruff+mypy clean, CI 4/4. Docs synced (fusion-plan P1?DONE, CODE_MAP person_detector entry + yolo_hybrid-as-orchestration, NEXT_STEPS item 2 ticked). ?? **NEXT (owner-steered):** **PR 3 = fusion P2.** The cheap, highest-confidence, PURE-LOGIC piece = **wire Gate A (`min_crossings`) into the live trajectory path** (`trajectory_hybrid`/`wasb_hybrid`) + add the knob to `IndexingConfig` ? *fixes the owner's held-shuttle false positive*, default OFF, immediately A/B-able via the harness on the 6 golden trajectories. Gate B (court-mask + player-count) is **owner-gated** on the court-mask-source decision (MULTI_SIGNAL_FUSION_PLAN §9.1: manual polygon vs auto homography ? owner). **P3 learned fusion + the SxS-on-new-videos RUN need GPU/owner-side** (persist person features on real video ? train on the 6 golden ? `compare_unlabeled(CONTROL, +person)` on new footage). Two PRs this session within [[working-agreement-merging]] self-merge authority.

Checkpoint: 2026-06-13 (EXPERIMENT HARNESS P1 SHIPPED ? **PR #114 MERGED**, master `ecee786`; suite 421?440 green). Picked up the rally-quality track (the desktop-agent pick-up `NEXT_STEPS` flagged; both design docs `EXPERIMENT_HARNESS.md` + `MULTI_SIGNAL_FUSION_PLAN.md` arrived via the session-start `git pull` 15e4e0e?9231af2, PR #112). Built `backend/eval/experiment.py` = the thin variant/experiment layer from `EXPERIMENT_HARNESS.md` P1 (the audit's "~80% already exists" HELD ? verified file:line; pure glue over metrics/calibration/classifier/training/regression.gt_hash/ gemini_refine.merge_overlaps/query_candidate_segments, **NO new scoring math**): `Variant` (JSON recipe: segmenter+config_overrides+cue_flags+Gate-A `min_crossings`+Gate-B `min_players`+learned classifier; `spec_hash`; pinned `CONTROL`), `compare` (labeled A/B, **LOVO-honest** train-on-others for learned variants, B?A deltas + `label_set_hash`), **`compare_unlabeled`** (the owner's **SxS-on-NEW-videos** ask ? agreed/a_only/b_only via match_intervals, no GT), `record_experiment` (? `eval_baselines/experiment_leaderboard.json`), `load_video_candidates` (DB bridge). 19 tests incl. the load-bearing **no-regression invariant** (all cues OFF ? byte-identical to CONTROL); ruff+mypy clean; CI 4/4. Docs synced (EXPERIMENT_HARNESS P1?DONE, CODE_MAP entry + stale regression-baseline-path fix, NEXT_STEPS item 3 ticked). Owner answered ramp-up Qs: **corpus = all 6** golden videos (he said "5"; LargeTestVideo/TestLargeVideo may be cuts of one match ? kept all 6 per the manifest); **merge authority = self-merge on green then continue to PR 2** (within [[working-agreement-merging]]). ?? NEXT (this session): **PR 2 = fusion P1** person/vision-cue extraction (lift torchvision+ByteTrack+velocity out of `yolo_hybrid` ? `backend/pipeline/detectors/person_detector.py` producer; yolo_hybrid consumes it, no behavior change ? regression-safe). Then P3 learned fusion trains on the 6 golden videos + `compare_unlabeled(CONTROL,+person)` SxS on new footage (GPU/owner-side). Tracks [[candidate-segments-finetuning]] + [[collector-indexer-bridge]].

Checkpoint: 2026-06-13 (WORKING-TREE CLEANUP ? master `15e4e0e`, tree now CLEAN). Cleaned up the leftover uncommitted bits (the cache-hygiene CODE was already MERGED as #109/#110 per the checkpoint below ? nothing to rescue there): (1) `config.json` GPU-run knobs **reverted to committed defaults** (was wasb_hybrid/7200/8 ? gemini/1800/1; values stay recorded in the cache-hygiene checkpoint below ? re-apply `default_segmenter: wasb_hybrid` in one line if that's the intended new default; user didn't answer keep-vs-revert, took the safe recoverable path). (2) stray root `arial.ttf` (unused ? shipped watermark font is committed `backend/assets/fonts/RobotoSlab-Regular.ttf` from #103) ? moved to `scratch/`. (3) genuine untracked draft `docs/STORAGE_SHARING_MODEL.md` (190-line storage-anchored monetization model, owner co-design pending) ? branched + **PR #111 OPEN (owner-gated, docs-only ? do NOT auto-merge**; supersedes COMMERCIALIZATION.md framing if ratified ? ADR then). Master tree now shows ONLY intentional carryovers (untracked `scratch/` handoffs+repro scripts, `artifacts/` Gen-0 bundle) ? could be gitignored for a pristine `git status` if desired (offered, not done). Reminder: `python -m tools.cleanup_caches --apply` reclaims the 96.5 GB the SessionStart hook flagged.

Checkpoint: 2026-06-13 (CACHE HYGIENE shipped ? **PR #109 + docs #110 MERGED**, master `15e4e0e`; + the ?5 reel + manual cleanup). **#110** = diligence docs follow-up (README §Cache Hygiene + knobs + tools/ line; CODE_MAP detector self-clean, tools/cleanup_caches.py entry, common-task rows; min_shot_count API-parity note) ? docs-only, CI 4/4 green. Both repos now have **0 open PRs**. After the GX010125 E2E run left ~163 GB in WSL `~/clips`, built a full cache-hygiene system (all on master): (1) **self-cleaning runners** ? `indexing.{wasb,tracknet}.keep_frames` (default false) deletes the frame cache + staged video after a SUCCESSFUL run; on failure keeps them for resume; (2) **tools/cleanup_caches.py** dry-run-first ops tool (`--apply`/`--keep-video`/`--older-than`/`--include-wasbcache`/`--summary`); (3) **disk guard** `min_free_gb`; (4) **SessionStart nudge hook** in `.claude/settings.local.json` (local, gitignored). Suite 409?**421 green**, ruff+mypy clean. Policy saved ? [[cache-hygiene-policy]]. ? WSL-scan footgun: the owner's login shell mangles `for e in *` loops (loop var empty) ? use du/find arg-globbing. Also this session (earlier): merged **#107** (API min_shot_count) + **#108** (watermark Windows colon fix); ran GX010125 E2E on GPU ? **GX010125_highlights.mp4** (11

rallies >7 shots) +
GX010125_highlights_min5.mp4 (24 rallies ?5, watermark now works ? validates #108) via a stitch-only reuse script
(no re-inference); confirmed **Gemini got 1080p** (4K only at final stitch); delivered an execution flowchart. Manual
WSL cleanup freed ~68 GB (kept GX010125's 93 GB frames + proxy per owner choice). ? LOCAL uncommitted `config.json`:
default_segmenter=wasb_hybrid / min_shot_count=8 / wasb.timeout_sec=7200 from the run. See [[gotcha-long-runs-gpu-wsl]].

Checkpoint: 2026-06-13 (SYNC + HANDOFF REFRESH ? read-only reconciliation by a ramp-up session; no new code from
me this pass). `git pull --ff-only` = already up to date at **master `6de70aa` (#108)**. Reconciled remote vs my last
session (#100): seven PRs landed from OTHER slates (NOT my work ? facts from git/gh): **#101** configurable branding
watermark · **#102** thread config stitching paddings into live VideoCompiler sites · **#103** bundle Apache-2.0
default watermark font + fallback log · **#104** C13 field kit (flyer/playbook/answer-sheet/signals) · **#105** i18n
design plan (Kannada-first) · **#107** enforce `stitching.min_shot_count` on /api/compile · **#108** escape Windows
drive-letter colon in ffmpeg filtergraph (fixes #103 watermark on Windows). **OPEN: #106** docs/video-locality-model
(awaiting owner). ? **UNCOMMITTED WIP IN TREE (sibling/owner ? DO NOT CLOBBER):** a WSL **cache-hygiene** slice in
progress, built on my `WslCommandMixin` ? `base.py` (`_wsl_free_gb`/`_wsl_rm_rf`), `wasb_runner.py`+`tracknet_runner.py`
(`keep_frames`/`min_free_gb` config + pre-run disk guard + post-success cache delete), `models.py` (new fields),
`config.json` (the GPU-run knobs: default_segmenter=wasb_hybrid, min_shot_count=8, wasb.timeout_sec=7200), untracked
`arial.ttf` (watermark font) + `docs/STORAGE_SHARING_MODEL.md`. Not yet PR'd by whoever owns it. [[session-handoff]]
rewritten to this state.

Checkpoint: 2026-06-13 (FIRST FULL E2E GPU RUN on a real 4K GoPro match + 2 merged fixes).
Owner asked to run the indexer E2E on the GPU over `F:\GoPro Hero Black 12\KhelSutra\Badminton 12
June 2026\GX010125.MP4` (12.8-min 4K60, 11.5GB), highlights for rallies with >7 shots, config-driven.
- **DELIVERED:** `output\GX010125_highlights.mp4` ? **11 rallies (shot_count 8?16) of 52 detected**;
  true 4K 3840×2160 h264 59.94fps, 2:23, 1.15GB; stitched from the 4K ORIGINAL (proxy only for indexing).
  wasb_hybrid: 64.8% shuttle visibility, 18 candidate windows ? 52 Gemini-confirmed rallies. min_shot_count=8
  correctly excluded a 7-shot rally (">7" semantics). Gemini ran clean (no 503s); ~47-min wall clock.
- **2 PRs merged on green CI** (owner's [[working-agreement-merging]] standing authorization):
  #107 `/api/compile` now enforces `stitching.min_shot_count` (parity w/ run_process_and_stitch);
  #108 stitcher `_ff_quote` escapes the Windows drive-letter colon ? fixes the on-by-default
  bundled-font watermark (PR #103) silently failing on Windows (`fontfile='C:/...'` broke ffmpeg
  drawtext parsing; also fixed absolute logo `movie=` paths). Both with regression tests; suite 409 green.
- **The hard-won run recipe is in [[gotcha-long-runs-gpu-wsl]]:** Modern Standby killed the first 2
  overnight attempts (keep-awake fix); cv2 PNG extraction (~38min) is the real bottleneck not the GPU
  (ffmpeg pre-extract, count-exact for CFR ? skips it); 1080p-proxy-index / 4K-stitch;

per-phase
339   observability (cache manifest for WASB, handoff dir for Gemini). Reusable scripts in `scratch/`
340   (run_gx010125.py, keep_awake.ps1, probe_*.py).
341   - ? LOCAL UNCOMMITTED `config.json`: default_segmenter=wasb_hybrid, min_shot_count=8, wasb.timeout_sec=7200
342   (set for this run; revert to gemini/1/1800 if not wanted). Watermark fix NOT applied to the delivered
343   reel (it ran before #108); a re-stitch would hit the DB segment cache + now-working watermark (~5min).
344
345   **Checkpoint:** 2026-06-12 (latest ? PAID COMPONENT REMOVED + VIDEO-LOCALITY MODEL; **PR #104 updated `9c84fea` +
346   PR #106 OPEN `85abe26`, both CI green, both AWAITING OWNER**). Owner spec: (1) remove ALL associate-pay from the
347   field kit ("something else in mind") ? flyer/playbook now "Engagement terms: discussed individually"; honesty
348   mechanics reworded ("judged on honest capture, not positive answers"); KEPT: WTP research questions (study subject)
349   + the ACADEMY's Phase-2 session fee (different party ? flagged, owner may strip too). (2) Storage/upload questions
350   answered from code: uploads ARE stored in full (`security.upload_dir`, NO retention policy ? BACKLOG); 10GB CANNOT
351   upload (max_upload_mb=4096 rejects at init; ?90MB sequential parts; hours on Indian uplinks). (3) NEW
352   `docs/VIDEO_LOCALITY_MODEL.md` (#106): central insight = timestamps are the product, video is dead weight (only the
353   compiler touches original bytes, cuttable where the original lives); locality ladder L0?L4 (L2 proxy-upload reuses
354   the collector ADR-002 frame-identical contract; L1-direct = client?Gemini, zero bytes through us; L0 = owned-model
355   north star; pilot = L4 sneakernet); **frugality frame (owner): bills affordable iff GPU doesn't burn ? L2/L1 serving
356   is GPU-less by construction, Gemini ¢/hr, GPU stays on the owned 2070S, per-video cost ceiling required**;
357   **academy-visit dividend: Phase-2 consented recordings ARE the multi-court target corpus, show-back = live L2 dress
358   rehearsal, demo reactions = build backlog**; 10 open questions OQ1?OQ10 (retention, MD5 identity, Gemini consent,
359   key custody, packaging, review source, reel delivery, uplink data, volume, L3 survival); rec lean = L2 default
360   (NOT a greenlight). NB: #106's base commit was pushed by owner/sibling mid-session (shared checkout). ?? owner:
361   review/merge #104 + #106; decide academy-fee keep/strip; the "something else" comp structure is his to define.
362
363   **Checkpoint:** 2026-06-12 (latest ? C13 KIT REFRAMED PRODUCT-FEEDBACK-FIRST per owner spec + **RECRUITING FLYER
364   added; PR #104 UPDATED `985a329`, CI 4/4 green, AWAITING OWNER REVIEW** ? do NOT merge without him). Owner spec:
365   hire someone to visit academies, talk to coaches AND students (how does it help + improvement feedback); stretch =
366   record ? pipeline ? show the academy ITS OWN sample result; UI/hosting "ready by then". Kit now = 4 docs by audience:
367   **`PRODUCT_FIELD_ASSOCIATE_FLYER.md`** (NEW ? candidate-facing recruiting flyer, owner fills ?/phone + recruits) ·
368   **`FIELD_VISIT_PLAYBOOK.md`** (RENAMED from FIELD_RESEARCH_ASSOCIATE_FLYER ? day-one playbook: demo-after-Q6 rule,
369   adult-18+-students-only w/ coach-points-out gating, Phase-2 show-back + scripted "can we keep using it?" no-promise
370   reply) · answer sheet (+demo-reaction/student blocks) · PMF_FIELD_SIGNALS (verdict machinery UNCHANGED; demo/student
371   data = product feedback only). 2nd adversarial critique (`w5aexakz2`, 30 agents): 12 confirmed+fixed (key honesty
372   fixes: "web app IS opening"?"we're building?will be able to try"; "match report"?"simple

summary"; "finds EVERY rally" dropped; minors loophole in "senior/adult students" closed), 15 refuted. ?? owner: review #104 (thresholds §3 + flyer copy) ? merge ? fill ?/phone ? trim 60s demo reel ? recruit. Owner ratified in-chat: **PMF validation = physically visiting Bangalore academies and reading signals.** Built the decision layer on top of the existing `docs/FIELD_RESEARCH_ASSOCIATE_FLYER.md` (which IS the "recruit hand-out" the owner remembered; the khelacharya PDFs in OneDrive are the separate internship flyer ? see [[sibling-internship-repo]]): **`docs/PMF_FIELD_SIGNALS.md`** (H1-H7 ? script-question map, G/A/R anchors, H5 counting rule = pre-ladder-or-budget-converted ??300 only ? anchored ?300 NEVER counts, pinned stopping rule n=15 target/logistics-only/verdict-once, numeric verdicts PROCEED / PIVOT A record+index / PIVOT B partner-API (PB Vision×PodPlay footnote now in-repo) / PIVOT C parent-payer / FALSIFIED) + **`docs/FIELD_VISIT_ANSWER_SHEET.md`** (printable per-visit form: verbatim answers, required most-negative quote, before/after-ladder tick, OFFICE-USE score grid) + flyer Q5 rewritten open-ended-first (old wording read the ?100/300/500 ladder ALOUD ? center-bias manufactures the exact ?300 PROCEED threshold; caught by the 38-agent adversarial critique `ww0rvaqf3`: 1 high/2 med/ 2 low confirmed+fixed, 9 refuted) + Stupa added to listen-for + NEXT_STEPS item 5 = kit COMPLETE. ?? owner: review #104 thresholds ? merge ? fill flyer ?/phone placeholders ? recruit the associate ? first 2 visits paired. PDFs of flyer+answer-sheet: generate AFTER placeholders are filled (offer stands).

Checkpoint: 2026-06-12 (latest ? ROORA VENDOR LOOP COMPLETE: **collector #23+#24+#25 MERGED** sequentially on green CI, owner pre-authorized; master `ca61405`, 0 open PRs, only owner's `.gitignore` edit uncommitted). **#23 boundary_grid_snap BLOCKER** in detect_issues (gcd-of-boundary-frame-diffs > ±0.3s bar ? gate BLOCKED; skips CSV-path/no-fps/<3 rallies) ? ? **replayed against the REAL 6-June PB+TT pilot zips: both auto-block with "All 12 rally boundaries sit on a 30-frame grid (~0.50s @ 59.94fps)"** (the manual forensics, now automated) + ROORA_BRIEF Phase-B revision (§7 REQUIRED method: CVAT step 1, first/last box on exact contact/dead frame, sparse interior OK; §4 grid-snap auto-detected warning; §2 skip-rallies-in-progress-at-file-edges; §1 annotation-optimized-copy note). **#24 prep-annotation-batch**: mirrors `<input-dir>/<match-folder>/<video>` ? proxies-only tree w/ same layout (thin orchestrator over the hardened single flow; per-video isolation; resumable "have" skips; live 2-court E2E smoke green). **#25 ROORA_RUNBOOK.md** (docs/annotation/): operator E2E ? send checklist (layout?batch?manifest commit?intake+email w/ the 2 non-negotiables), receive (gate decision table incl. grid-snap, review sheet, import/export on ORIGINALS), troubleshooting table for every guard. Suite **180 green**. ?? Vendor loop is now fully tooled+documented. ?? AGREED NEXT (owner, session wrap 2026-06-12): (1) owner fills the canonical doc's TODO(avidullu) markers + screenshots, labels it **v8**; (2) **joint TEST RUN of the full vendor loop on real footage BEFORE anything goes to Roora** (runbook §1 send-side end-to-end: batch layout ? prep-annotation-batch ? manifest ? mock-receive via validate-delivery on an old delivery); (3) only then send. **v7 ANOMALY RESOLVED (2026-06-12):** the canonical vendor-facing Brief+Guide is the Google Doc at docs.google.com/document/d/1NvAXE0q7t4Z0QddLkEfnta0DU982yDZaqkGNPMe8EY0 (provenance: generated from master `ca61405`; supersedes Drive v3/v5/v6/v7; owner confirmed superset-of-v7) ? see [[roora-guidelines-drive-doc]]; before sending: pick ONE version label + fill its TODO(avidullu) markers/screenshots.

Checkpoint: 2026-06-12 (earlier ? ADR-002 IMPLEMENTED: **collector #22 MERGED** `a068fd1`, owner authorized merge-on-green; #21 ADR + #20 + indexer #102/#103 all merged earlier today, slate was clean). Shipped: `curate prep-annotation-proxy` + `src/core/proxy.py` ? ?1080p/`-g 15`/CRF 19/audio-kept re-encode w/ `-fps_mode passthrough` (NEVER `-r`), post-encode parity verify (exact frame count; fps ±0.1%; duration like-with-like stream/container), delete-on-ANY-failure (incl. Ctrl-C), VFR refusal w/o `--allow-vfr`, provenance ? `data/annotation_proxy_manifest.csv` (source+proxy MD5s; video_id collision guard BEFORE encode; Excel-lock prints row + keeps proxy), same-file guard normcase/realpath/samefile (NTFS case-variant `--out`+`--force` would have let `ffmpeg -y` TRUNCATE the source ? review verifier reproduced it live pre-fix). Suite **169 green** (+32), CI green. Verified live on ffmpeg 8.1.1 (smoke clip 59.94fps: parity 179 frames both files; smoke caught `-vsync` deprecation ? `-fps_mode`). Process: ultracode adversarial review workflow (`wyv8524zf`, 18 agents) confirmed 1 high/2 med/7 low ? all fixed pre-merge. Docs: INGESTION_PIPELINE step 0 + diagram leg + onboarding FAQ + sampling-FAQ split (highlights-vs-golden), ADR-002 provenance bullet as-implemented amendment, PIPELINE_ROLE ADR refs qualified. ?? REMAINING ADR-002 follow-ups (separate items, NOT started): Phase-B brief revision (frame step 1 / exact boundary frames / "we supply proxies") ? owner words vendor comms; grid-snap QA check in validate-delivery. First real use: run prep-annotation-proxy on the next Roora batch before handoff.

Checkpoint: 2026-06-12 (earlier ? ANNOTATION-PROXY DECISION RATIFIED ? **collector PR #21 OPEN**, docs-only **ADR-002** in collector `docs/DECISIONS.md`). Owner agreed in-chat w/ my rec from the assessment checkpoint below: proxies generated in-repo via a future `curate prep-annotation-proxy`. ADR records the proxy contract (SAME fps ? never reduce; no trim; frame-count parity check; 1080p/CRF18-20/-g15/audio kept; intake-manifest row w/ original+proxy MD5s), the why-collector rationale (owns the vendor flow = command #0 of convert-cvat?validate-delivery?import-local; verification+provenance is the durable 80%, one-off scripts skip it), and the **Phase-B necessity argument the owner wanted captured: ~12-15 videos/sport × ~30 min 4K/59.94 = >100k frames/file ? vendor frame-step-1 annotation infeasible without proxies ? near-precondition, not optimization**. Follow-ups recorded IN the ADR: Phase-B brief convention line (step=1, first/last box on exact contact/dead frame) + re-add grid-snap QA to validate-delivery. Collector checkout back on master (owner's stray `.gitignore` edit untouched). ? sibling PR #20 (below) edits rally_cvat.py docstring rationale 0.5/1.0?0.5/0.5 ? no file overlap w/ #21, both docs-true. ?? owner reviews/merges #21 (+ #102/#20 below); subcommand IMPLEMENTATION stays owner-gated (separate PR on greenlight).

Checkpoint: 2026-06-12 (stitching-padding wiring fix ? **indexer #102 + collector #20 OPEN, CI green, awaiting owner review**). Audit finding (d) (2026-06-10) fixed: config.json `stitching.{begin,end}_padding` was silently ignored by the live compiler ? both `backend/main.py` VideoCompiler sites (`/api/compile` + CLI trim) passed only `watermark=`, so reels always used ctor defaults 0.5/**1.0** vs configured 0.5/0.5. Fix

```
threads the typed
StitchingConfig paddings into both sites (+ `StitchingConfig` re-export from backend.config); regression test drives
the REAL compiler through /api/compile (mocked ffmpeg) asserting a config override (2.0/3.0) reaches `-ss/-to`
(8.0/17.0 for a 10?14s segment ? unwired defaults would give 9.5/15.0); CODE_MAP truth-synced (ctor-vs-config
distinction + recipe row now cites the main.py wiring). ? **Behavior change on merge: live API reels end padding
1.0?0.5s** (the config value). Companion **collector #20** = docstring-only: `rally_cvat.py::detect_issues` cited the
hardcoded 0.5/1.0 + a stale module path; now cites config-driven 0.5/0.5 (rationale still holds; ? cross-checks the
sibling Roora-proxy thread below ? boundary-slack rationale now assumes 0.5s end padding, not 1.0s). Verified: indexer
suite **401 green** + 4/4 CI lanes; collector **137 green** + CI. Both repos back on master; owner's uncommitted
collector `.gitignore` edit untouched. NOTE: untracked `arial.ttf` at indexer root (not mine ? left alone, likely a
watermark-font experiment). ?? owner reviews/merges #102 + collector #20.
```

7. assistant (2026-06-14T09:49:43.199Z)

Let me ramp up on the eval harness internals and a sample golden JSON. I'll read the key files in parallel.

8. user (2026-06-14T09:49:44.170Z)

```
"""Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden set.

The fusion-P3 measurement. Loads the per-video feature JSONs written by `fusion_golden.py`
and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-out) ?
no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the harness's
designed seam): the person features are extra columns the learned LogisticRegression weights,
so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.

Run (no GPU):
    python -m backend.eval.fusion_compare --features-dir output --record
"""
from __future__ import annotations

import argparse
import glob
import json
import os
from typing import Any, Dict, List

from backend.eval import experiment
from backend.eval.eval_stats import paired_delta_ci
from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES


def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
    """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign test
    (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out scores
    by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-videos."""
```

```python
    by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
    by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
    vids = [p["video"] for p in res["variant_a"]["per_video"]]
    return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])


def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
    """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
    feature columns + golden gts). The harness derives labels from gts itself."""
    videos: List[experiment.VideoCandidates] = []
    for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
        with open(path, encoding="utf-8") as f:
            d = json.load(f)
        cands = d["candidates"]
        intervals = [(float(c["start"]), float(c["end"])) for c in cands]
        features: Dict[str, List[float]] = {}
        for fn in SHUTTLE_FEATURE_NAMES:
            features[fn] = [float(c["shuttle"][fn]) for c in cands]
        for fn in PERSON_FEATURE_NAMES:
            features[fn] = [float(c["person"][fn]) for c in cands]
        gts = [(float(a), float(b)) for a, b in d["gts"]]
        videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
                                                 features=features, gts=gts))
    return videos


def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
            threshold: float = 0.5) -> Dict[str, Any]:
    shuttle_only = experiment.Variant(name="shuttle_only", feature_names=list(SHUTTLE_FEATURE_NAMES),
                                      threshold=threshold)
    fusion = experiment.Variant(name="shuttle_person_fusion",
                                feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),
                                threshold=threshold)
    return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)


def format_result(res: Dict[str, Any], iou: float) -> str:
    a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
    ds = delta_stats(res)
    st = ds["sign_test"]
    lines = [
        f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest) ===",
        f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}  IoU={a['mean_iou']:.3f}",
        f"  +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}  IoU={b['mean_iou']:.3f}",
        f"  ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)  ?P={d['precision']:+.3f}  ?R={d['recall']:+.3f}",
        f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f}, {ds['ci_high']:+.3f}]  "
        f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
        f"| CI clears 0: {ds['ci_excludes_zero']}",
        "  per-video ?F1:",
    ]
    for pv in res["per_video_delta"]:
        lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
    return "\n".join(lines)


def main() -> None:
    ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs shuttle+person (golden set).")
```

```
        ap.add_argument("--features-dir", default="output")
        ap.add_argument("--iou", type=float, default=0.5)
        ap.add_argument("--record", action="store_true", help="append the result to the experiment leaderboard")
        args = ap.parse_args()

        videos = load_videos(args.features_dir)
        if len(videos) < 2:
            raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found {len(videos)})")
        res = compare(videos, iou=args.iou)
        print(format_result(res, args.iou))
        if args.record:
            row = experiment.record_experiment(res)
            print(f"recorded -> eval_baselines/experiment_leaderboard.json ({row['timestamp']})")


    if __name__ == "__main__":
        main()

```

9. user (2026-06-14T09:49:44.659Z)

```
"""Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-only).

Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment` runs
`detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio features per
window; `compare_increment` then runs the SHIPPED harness between a shuttle+person variant
and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON TOP
of person, not against raw shuttle. Audio enters only as columns the learned scorer weights
(no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.

Run (no GPU):
    python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json --features-dir output --augment --compare
"""
from __future__ import annotations

import argparse
import glob
import json
import os
from typing import Any, Dict, List

from backend.eval import experiment
from backend.eval.fusion_compare import delta_stats
from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES, _derive_video_path
from backend.pipeline.detectors import audio_features as af
from backend.pipeline.audio.hit_detector import detect_hits_in_video


def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
    """Add the 3 audio features to each existing ``<name>_golden_features.json`` window."""
    with open(manifest_path, encoding="utf-8") as f:
        specs = json.load(f)
    written: List[str] = []
```

```
    for spec in specs:
        jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
        if not os.path.isfile(jp):
            print(f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden first; skipping")
            continue
        video = _derive_video_path(spec)
        print(f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...", flush=True)
        hits = detect_hits_in_video(video, k=k)
        with open(jp, encoding="utf-8") as f:
            data = json.load(f)
        for c in data["candidates"]:
            c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
        with open(jp, "w", encoding="utf-8") as f:
            json.dump(data, f, indent=2)
        print(f"[fusion_audio] {spec['name']}: {len(hits)} hits over {len(data['candidates'])} windows", flush=True)
        written.append(jp)
    return written


def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
    """Load the feature JSONs into VideoCandidates including the audio columns (requires
    `augment` to have run ? raises if a JSON lacks audio)."""
    videos: List[experiment.VideoCandidates] = []
    for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
        with open(path, encoding="utf-8") as f:
            d = json.load(f)
        cands = d["candidates"]
        if cands and "audio" not in cands[0]:
            raise SystemExit(f"{path} has no audio features ? run `--augment` first")
        intervals = [(float(c["start"]), float(c["end"])) for c in cands]
        features: Dict[str, List[float]] = {}
        for fn in SHUTTLE_FEATURE_NAMES:
            features[fn] = [float(c["shuttle"][fn]) for c in cands]
        for fn in PERSON_FEATURE_NAMES:
            features[fn] = [float(c["person"][fn]) for c in cands]
        for fn in af.AUDIO_FEATURE_NAMES:
            features[fn] = [float(c["audio"][fn]) for c in cands]
        gts = [(float(a), float(b)) for a, b in d["gts"]]
        videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
                                                 features=features, gts=gts))
    return videos


def compare_increment(videos: List[experiment.VideoCandidates], iou: float = 0.5,
                      threshold: float = 0.5) -> Dict[str, Any]:
    """Incremental audio lift: shuttle+person  vs  shuttle+person+audio (honest LOVO)."""
    person = experiment.Variant(
        name="shuttle_person",
        feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES), threshold=threshold)
    person_audio = experiment.Variant(
        name="shuttle_person_audio",
        feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES) + list(af.AUDIO_FEATURE_NAMES),
        threshold=threshold)
    return experiment.compare(videos, person, person_audio, iou=iou, honest=True)


def format_result(res: Dict[str, Any], iou: float) -> str:
    a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
    ds = delta_stats(res)
```

```
    st = ds["sign_test"]
    lines = [
        f"=== Fusion P4 incremental AUDIO LOVO (n={res['num_videos']}, IoU>={iou}, honest) ===",
        f"  shuttle+person       : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}",
        f"  shuttle+person+audio : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}",
        f"  incremental ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)",
        f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f}, {ds['ci_high']:+.3f}]  "
        f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
        f"| CI clears 0: {ds['ci_excludes_zero']}",
    ]
    for pv in res["per_video_delta"]:
        lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
    return "\n".join(lines)


def main() -> None:
    ap = argparse.ArgumentParser(description="Fusion P4 ? incremental audio lift on the golden set.")
    ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
    ap.add_argument("--features-dir", default="output")
    ap.add_argument("--k", type=float, default=2.5, help="hit-detector sensitivity")
    ap.add_argument("--augment", action="store_true", help="add audio features to the existing JSONs")
    ap.add_argument("--compare", action="store_true", help="report the incremental audio LOVO ?F1")
    ap.add_argument("--iou", type=float, default=0.5)
    ap.add_argument("--record", action="store_true")
    args = ap.parse_args()

    if args.augment:
        augment(args.manifest, args.features_dir, args.k)
    if args.compare:
        videos = load_videos_with_audio(args.features_dir)
        if len(videos) < 2:
            raise SystemExit(f"need >=2 augmented feature files in {args.features_dir}")
        res = compare_increment(videos, iou=args.iou)
        print(format_result(res, args.iou))
        if args.record:
            row = experiment.record_experiment(res)
            print(f"recorded -> eval_baselines/experiment_leaderboard.json ({row['timestamp']})")


if __name__ == "__main__":
    main()

```

10. user (2026-06-14T09:49:45.709Z)

```
"""Experiment harness ? A/B + shadow + side-by-side for rally detection.

This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
no-regression gate, the pinnable classifier artifact, the persisted-candidate
substrate) already exists ? this module composes it. **No new scoring math lives
here**; everything defers to:

  - `backend.eval.metrics`        ? `score`, `match_intervals`, `interval_iou`
  - `backend.eval.training`       ? `label_candidates` (y by the same IoU matcher)
  - `backend.eval.classifier`     ? `LogisticRegression` (the pinnable JSON model)
  - `backend.eval.gemini_refine`  ? `merge_overlaps` (the blessed window merge)
```

```
  - `backend.eval.regression`     ? `gt_hash` (label-set fingerprint)
  - `backend.eval.calibration`    ? `pct_improvement`

The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
(per-frame signals ? run once on the GPU/WSL box, persisted into
`candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
given persisted candidate features it produces `List[Interval]` deterministically,
with no GPU and zero production risk. So most experiments are pure offline
re-scoring of stored data and never touch the served pipeline.

Three modes (`EXPERIMENT_HARNESS.md` §5):
  - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
    aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
    held-out loop, but variant-parameterised.
  - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
    variants agree / diverge (A-only, B-only, agreed). The divergences are what a
    human spot-checks (and what two highlight reels would show side by side).
  - the promotion gate stays `run_regression.py` + `regression_baseline.json`
    (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**

Pure + offline + unit-tested. The only DB-touching helper is
`load_video_candidates`, which reads persisted candidates via
`Database.query_candidate_segments`.
"""
from __future__ import annotations

import json
import logging
import os
from dataclasses import dataclass, field
from datetime import datetime, timezone
from hashlib import sha1
from typing import Any, Callable, Dict, List, Optional, Sequence

from backend.eval.calibration import pct_improvement
from backend.eval.classifier import LogisticRegression
from backend.eval.gemini_refine import merge_overlaps
from backend.eval.metrics import Interval, match_intervals, score
from backend.eval.regression import gt_hash
from backend.eval.training import label_candidates
from backend.eval import eval_stats as _stats           # C1: CIs + paired sign test
from backend.eval import segmentation_metrics as _segm   # C4: F1@k + segment-count ratio
from backend.eval.score_calibration import fit_isotonic, fit_platt   # C2: calibrate cue scores

logger = logging.getLogger(__name__)

# Where a comparison run appends one reproducible record. Tracked location (NOT under
# the gitignored output/) so the leaderboard survives a clean checkout, mirroring
# regression.DEFAULT_BASELINE_PATH.
DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
_METRICS = ("precision", "recall", "f1", "mean_iou")


class ExperimentError(ValueError):
    """A variant cannot be evaluated as configured (e.g. a learned variant asked to
    score an unlabeled video with no pinned model, or a gate referencing an absent
    feature)."""


# -------------------------------------------------------------------------- Variant


```

```
@dataclass
class Variant:
    """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).

    A variant turns one video's persisted candidate windows + features into rally
    intervals. Every knob defaults to the **control** behaviour (no gate, no learned
    filter), so a variant that merely *carries* a new, disabled cue is byte-identical
    to control ? the core no-regression guarantee.

    Fields:
      - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
        for the leaderboard and for selecting the served path (the existing
        `segmenter_registry` + `IndexingConfig` do the actual selection in production).
        New cues are recorded here default-OFF.
      - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
        ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
        `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-shuttle
        windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
      - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
        whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
        cue persists that feature).
      - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
        `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
        required for `compare_unlabeled` on a learned variant.
      - ``feature_names`` ? the persisted features the learned filter consumes. When set
        WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
        (honest) ? the recipe, not a pinned artifact.
      - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
        overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
    """

    name: str
    segmenter: str = "wasb_hybrid"
    config_overrides: Dict[str, Any] = field(default_factory=dict)
    cue_flags: Dict[str, bool] = field(default_factory=dict)
    min_crossings: int = 0
    min_players: int = 0
    classifier_model: Optional[str] = None
    feature_names: Optional[List[str]] = None
    threshold: float = 0.5
    merge_gap: float = 1.0
    # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the operating
    # point is chosen on the TRAINING fold, never the held-out one ? fixing the first-A/B
    # failure where a fixed 0.5 zeroed out a small-candidate video.
    tune_threshold: bool = False          # pick the F1-best threshold on the train fold
    calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate proba first

    def is_learned(self) -> bool:
        """True if this variant applies a learned classifier (pinned model or recipe)."""
        return self.classifier_model is not None or bool(self.feature_names)

    def to_dict(self) -> dict:
        return {
            "name": self.name,
            "segmenter": self.segmenter,
            "config_overrides": self.config_overrides,
            "cue_flags": self.cue_flags,
            "min_crossings": self.min_crossings,
            "min_players": self.min_players,
            "classifier_model": self.classifier_model,
```

```
            "feature_names": self.feature_names,
            "threshold": self.threshold,
            "merge_gap": self.merge_gap,
            "tune_threshold": self.tune_threshold,
            "calibrate": self.calibrate,
        }

    @classmethod
    def from_dict(cls, d: dict) -> "Variant":
        known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
        return cls(**{k: v for k, v in d.items() if k in known})

    def spec_hash(self) -> str:
        """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
        and makes a result reproducible. The pinned **control** variant always hashes the
        same, so a regression is always traceable to a recipe change, never to drift."""
        blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
        return sha1(blob.encode()).hexdigest()[:12]


#: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
#: filter. Always the comparison reference; "rollback" = make this the served variant.
CONTROL = Variant(name="control")


# ----------------------------------------------------------------- persisted candidates


@dataclass
class VideoCandidates:
    """One video's persisted detection, ready for offline re-scoring.

    ``intervals`` are the candidate windows; ``features`` is column-major
    (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
    ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
    with `load_video_candidates`, or directly in tests.
    """

    name: str
    intervals: List[Interval]
    features: Dict[str, List[float]] = field(default_factory=dict)
    gts: Optional[List[Interval]] = None


def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
    """Persisted feature column, defaulting missing/short columns to 0.0 (matches
    `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
    col = vc.features.get(name)
    n = len(vc.intervals)
    if col is None:
        logger.warning("variant references feature %r absent from %s ? treating as 0.0",
                       name, vc.name)
        return [0.0] * n
    if len(col) != n:
        raise ExperimentError(
            f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
    return [float(x) for x in col]


def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) -> List[List[float]]:
    cols = [_feature_column(vc, name) for name in feature_names]
    return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]

```

```
def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
    """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B players)."""
    n = len(vc.intervals)
    mask = [True] * n
    if variant.min_crossings > 0:
        col = _feature_column(vc, "net_crossings")
        mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
    if variant.min_players > 0:
        col = _feature_column(vc, "players_in_court")
        mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
    return mask


def predict(variant: Variant, vc: VideoCandidates,
            model: Optional[LogisticRegression] = None,
            threshold: Optional[float] = None,
            calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
    """Variant prediction: gate by persisted features, optionally filter by a learned
    model, then merge. Pure and deterministic.

    ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
    fold; if omitted and the variant pins ``classifier_model``, that frozen model is
    loaded. A learned variant with neither a supplied nor a pinned model raises ? there
    is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
    fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies them.
    """
    mask = _gate_mask(variant, vc)
    if variant.feature_names:
        if model is None:
            if variant.classifier_model is None:
                raise ExperimentError(
                    f"variant {variant.name!r} is learned but has no model to predict "
                    f"with (pass a fitted model or set classifier_model)")
            model = LogisticRegression.load(variant.classifier_model)
        proba = [float(p) for p in model.predict_proba(_feature_matrix(vc, variant.feature_names))]
        if calibrator is not None:
            proba = [calibrator(p) for p in proba]
        thr = variant.threshold if threshold is None else threshold
        mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
    kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
    return merge_overlaps(kept, gap_tol=variant.merge_gap)


def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                        train_videos: Sequence[VideoCandidates], iou: float
                        ) -> "tuple[Optional[Callable[[float], float]], Optional[float]]":
    """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
    (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
    (use the variant default). Honest: never looks at the held-out video.
    """
    calibrator: Optional[Callable[[float], float]] = None
    if variant.calibrate:
        raw: List[float] = []
        y: List[int] = []
        for vc in train_videos:
            raw.extend(float(p) for p in
                       model.predict_proba(_feature_matrix(vc, variant.feature_names or [])))
            y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
        if variant.calibrate == "platt":
```

```
            calibrator = fit_platt(raw, y)
        elif variant.calibrate == "isotonic":
            calibrator = fit_isotonic(raw, y)
        else:
            raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use 'platt'/'isotonic')")
    threshold: Optional[float] = None
    if variant.tune_threshold:
        best_t, best_f1 = variant.threshold, -1.0
        for k in range(1, 20):                       # grid 0.05..0.95 on the train fold's rally F1
            t = round(0.05 * k, 2)
            f1s = [score(predict(variant, vc, model=model, threshold=t, calibrator=calibrator),
                         vc.gts or [], iou).f1 for vc in train_videos]
            mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
            if mean_f1 > best_f1:
                best_f1, best_t = mean_f1, t
        threshold = best_t
    return calibrator, threshold


# ---------------------------------------------------------------------- variant scoring


def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) -> LogisticRegression:
    """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via the
    evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
    X: List[List[float]] = []
    y: List[int] = []
    for vc in videos:
        if vc.gts is None:
            raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
        X.extend(_feature_matrix(vc, variant.feature_names or []))
        y.extend(label_candidates(vc.intervals, vc.gts, iou))
    if not X:
        raise ExperimentError("cannot train a learned variant on zero candidates")
    return LogisticRegression().fit(X, y, variant.feature_names)


def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
                     iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
    """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
    mean aggregate.

    Prediction policy:
      - **static variant** (no learned filter): predict each video directly ? no training,
        so no leakage; per-video scoring is already honest.
      - **learned, pinned model** (``classifier_model`` set): score every video with the
        SHIPPED model. ? optimistic if those videos were in its training set ? use this to
        report "how the shipped artifact does here", not for the promotion decision.
      - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO ?
        train on the OTHER videos, predict the held-out one. The number to trust.
      - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity only).
    """
    for vc in videos:
        if vc.gts is None:
            raise ExperimentError(f"{vc.name} has no golden labels; use compare_unlabeled")
```

```python

    per_video: List[Dict[str, Any]] = []
    predictions: Dict[str, List[Interval]] = {}

    recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
    pinned_model = (LogisticRegression.load(variant.classifier_model)
                    if variant.classifier_model is not None else None)
    all_model = None
    if recipe_lovo and not honest:
        all_model = _train(variant, videos, iou)

    for i, vc in enumerate(videos):
        cal: Optional[Callable[[float], float]] = None
        thr: Optional[float] = None
        if recipe_lovo and honest:
            others = [v for j, v in enumerate(videos) if j != i]
            if not others:
                raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a pinned model")
            model: Optional[LogisticRegression] = _train(variant, others, iou)
            if variant.tune_threshold or variant.calibrate:    # operating point from the train fold
                assert model is not None                       # _train never returns None
                cal, thr = _calibrate_and_tune(variant, model, others, iou)
        elif recipe_lovo:                      # honest=False ? one model over all videos
            model = all_model
        else:                                  # static variant or pinned model
            model = pinned_model
        preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
        assert vc.gts is not None              # guarded at function top
        sc = score(preds, vc.gts, iou)
        per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m) for m in _METRICS}})
        predictions[vc.name] = preds

    aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else 0.0)
                 for m in _METRICS}
    return {
        "variant": variant.name,
        "spec_hash": variant.spec_hash(),
        "is_learned": variant.is_learned(),
        "honest_lovo": recipe_lovo and honest,
        "per_video": per_video,
        "aggregate": aggregate,
        "predictions": predictions,
    }


def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
    """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare mean)."""
    return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in _METRICS}


def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) -> Dict[str, Any]:
    """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU hides)."""
    gts_by_name = {v.name: (v.gts or []) for v in videos}
    overlaps = _segm.DEFAULT_OVERLAPS
    f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
```

```python
    ratios: List[float] = []
    for name, preds in eval_v.get("predictions", {}).items():
        gts = gts_by_name.get(name, [])
        got = _segm.f1_at_overlaps(preds, gts, overlaps)
        for ov in overlaps:
            f1k[ov].append(got[ov])
        ratios.append(_segm.segment_count_ratio(preds, gts))

    def _mean(xs: List[float]) -> float:
        return sum(xs) / len(xs) if xs else 0.0
    out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in f1k.items()}
    out["segment_count_ratio"] = _mean(ratios)
    return out


def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
                   eval_b: Dict[str, Any]) -> Dict[str, Any]:
    """C1 + C4 honest reporting layered over a compare() pair (additive, EXPERIMENT_HARNESS §6).

    ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in order),
    so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real when
    the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
    """
    a_f1 = [p["f1"] for p in eval_a["per_video"]]
    b_f1 = [p["f1"] for p in eval_b["per_video"]]
    return {
        "variant_a_ci": _ci_block(eval_a),
        "variant_b_ci": _ci_block(eval_b),
        "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
        "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
                         "variant_b": _segmentation_block(videos, eval_b)},
    }


def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
            iou: float = 0.5, honest: bool = True, honest_stats: bool = True) -> Dict[str, Any]:
    """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the same
    golden corpus and reports the **B ? A** deltas, per video and in aggregate.

    The deltas use the existing `metrics.score` (per video) + `calibration.pct_improvement`;
    learned variants are scored LOVO-honestly. The result is a self-contained record fit for
    `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with it.

    ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap CIs, a
    paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ? *alongside* the
    existing bare-mean fields (additive: nothing existing changes). Set False for the legacy
    shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md` §13/C1+C4): a
    bare LOVO mean at n?6 is not trustworthy on its own.
    """
    if len(videos) < 1:
        raise ExperimentError("compare needs at least one labeled video")
    eval_a = evaluate_variant(videos, variant_a, iou, honest)
```

```
        eval_b = evaluate_variant(videos, variant_b, iou, honest)

        delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
        delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for m in _METRICS}

        by_video_a = {p["video"]: p for p in eval_a["per_video"]}
        per_video_delta = [
            {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
            for p in eval_b["per_video"]
        ]

        # One fingerprint over the whole label set ? detects "the deltas were measured against
        # different labels" the same way regression.gt_hash guards the no-regression baseline.
        all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]

        result: Dict[str, Any] = {
            "iou": iou,
            "label_set_hash": gt_hash(all_gts),
            "num_videos": len(videos),
            "variant_a": eval_a,
            "variant_b": eval_b,
            "delta": delta,
            "delta_pct": delta_pct,
            "per_video_delta": per_video_delta,
        }
        if honest_stats:
            result["honest"] = honest_summary(videos, eval_a, eval_b)
        return result


    # ---------------------------------------------------------------- SxS on unlabeled video


    def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
                          agree_iou: float = 0.5) -> Dict[str, Any]:
        """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).

        With no labels there is no lift to measure ? instead this surfaces where the two
        variants **agree vs diverge**, which is exactly what a human reviews (and what two
        highlight reels would show side by side):

          - ``agreed``  ? windows both variants found (matched at ``agree_iou``);
          - ``a_only``  ? A fired, B suppressed (B's added precision, if A was over-firing);
          - ``b_only``  ? B fired, A did not (B's new detections ? real recall or new FPs:
            the human / a later golden label adjudicates).

        Both variants must be predictable without labels: a learned variant therefore needs a
        pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
        `metrics.match_intervals` ? no new matching logic.
        """
        preds_a = predict(variant_a, video)
        preds_b = predict(variant_b, video)
        mr = match_intervals(preds_a, preds_b, agree_iou)

        agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
                  for m in mr.matches]
        agree_ious = [m.iou for m in mr.matches]
        a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
        b_only = [preds_b[i] for i in mr.unmatched_gt_indices]

        def _dur(ivs: List[Interval]) -> float:
```

```python
        return sum(e - s for s, e in ivs)

    return {
        "video": video.name,
        "agree_iou": agree_iou,
        "variant_a": variant_a.name,
        "variant_b": variant_b.name,
        "num_a": len(preds_a),
        "num_b": len(preds_b),
        "num_agreed": len(agreed),
        "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
        "duration_a": _dur(preds_a),
        "duration_b": _dur(preds_b),
        "agreed": agreed,
        "a_only": a_only,
        "b_only": b_only,
    }


# --------------------------------------------------------------------- leaderboard / DB


def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
                      timestamp: Optional[str] = None) -> Dict[str, Any]:
    """Append a compact, reproducible record of a `compare()` result to the JSON
    leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
    `regression_baseline.json`; deliberately the size of a baseline file, not a service
    (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
    """
    row: Dict[str, Any] = {
        "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
        "iou": result.get("iou"),
        "label_set_hash": result.get("label_set_hash"),
        "num_videos": result.get("num_videos"),
        "variant_a": {"name": result["variant_a"]["variant"],
                      "spec_hash": result["variant_a"]["spec_hash"],
                      "aggregate": result["variant_a"]["aggregate"]},
        "variant_b": {"name": result["variant_b"]["variant"],
                      "spec_hash": result["variant_b"]["spec_hash"],
                      "aggregate": result["variant_b"]["aggregate"]},
        "delta": result.get("delta"),
    }
    # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries the
    # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
    honest = result.get("honest") or {}
    if honest.get("paired_delta_f1"):
        row["honest_delta_f1"] = honest["paired_delta_f1"]
    entries: List[Dict[str, Any]] = []
    if os.path.isfile(path):
        with open(path, encoding="utf-8") as f:
            entries = json.load(f)
    entries.append(row)
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        json.dump(entries, f, indent=2)
    logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
                row["variant_a"]["name"], row["variant_b"]["name"],
                (result.get("delta") or {}).get("f1", 0.0), path)
    return row


def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
                          gts: Optional[List[Interval]] = None,
                          feature_names: Sequence[str] = ()) -> VideoCandidates:
```

```
    """Bridge persisted detection ? `VideoCandidates` via `Database.query_candidate_segments`.

    ``feature_names`` selects which persisted features to lift out of each row's ``stats``
    JSON (``"duration"`` is derived from the window bounds). This is the only DB-touching
    function; the comparison itself is pure and offline. Exercised owner-side on a real DB,
    but works against any `Database` (including an in-memory one in tests).
    """
    rows = db.query_candidate_segments(video_id, detector=detector)
    intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in rows]
    features: Dict[str, List[float]] = {}
    for name in feature_names:
        if name == "duration":
            features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in rows]
        else:
            features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in rows]
    return VideoCandidates(name=video_id, intervals=intervals, features=features, gts=gts)

```

11. assistant (2026-06-14T09:49:49.624Z)

Now I understand the harness architecture. Let me inspect a sample golden JSON to see which feature columns exist (shuttle/person, and whether audio is present yet), the feature name lists, the eval_stats module, and the existing baselines/manifest.

12. user (2026-06-14T09:49:54.081Z)

```
adarsh_avi_singles           n_cand=172 n_pos=80 n_gts=96 keys=['start', 'end', 'shuttle', 'person', 'label', 'audio'] audio=True
   shuttle feats: ['duration', 'peak_velocity', 'mean_velocity', 'active_ratio', 'net_crossings']
   person  feats: ['players_in_court', 'two_sided_motion', 'mean_player_motion', 'in_court_ratio', 'shuttle_player_colocation']
   audio   feats: ['audio_hit_count', 'audio_hit_rate', 'audio_hit_strength_mean']
```

13. user (2026-06-14T09:49:54.356Z)

```
"""Honest small-N evaluation statistics (course-corrections C1 + C3).

The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where a
single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's training
and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
turns those into an honest summary:

- **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
  interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
  many *videos* it wins on (the distribution-free test the repo already leans on:
  "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote noise.
- **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a learned
  producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
  match, never a held-out window from the same video).

```

```
See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib only),
deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
"""
from __future__ import annotations

import math
import random
import statistics
from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple

Interval = Tuple[float, float]
FitPredict = Callable[[List[int], List[int]], List[Any]]


def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
                 n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
    """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.

    ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread to
    resample. With n=6 (our corpus) this is the honest "how wide could the mean be?" band.
    """
    vals = [float(v) for v in values]
    n = len(vals)
    if n == 0:
        return (0.0, 0.0)
    if n == 1:
        return (vals[0], vals[0])
    rng = random.Random(seed)
    means: List[float] = []
    for _ in range(max(1, n_boot)):
        means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
    means.sort()
    alpha = (1.0 - confidence) / 2.0
    last = len(means) - 1
    lo = means[max(0, int(math.floor(alpha * last)))]
    hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
    return (lo, hi)


def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
                 n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
    """``{mean, std, n, ci_low, ci_high, confidence}`` ? report this, never a bare mean (C1)."""
    vals = [float(v) for v in values]
    n = len(vals)
    mean = statistics.mean(vals) if vals else 0.0
    std = statistics.pstdev(vals) if n > 1 else 0.0
    lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
    return {"mean": mean, "std": std, "n": float(n),
            "ci_low": lo, "ci_high": hi, "confidence": confidence}


def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
    """Two-sided exact sign test over per-fold deltas (B ? A).

    ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
    excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
    distribution-free "did B win on enough *videos*?" check.
    """
    wins = sum(1 for d in deltas if d > eps)
    losses = sum(1 for d in deltas if d < -eps)
```

```
    ties = len(deltas) - wins - losses
    n_eff = wins + losses
    if n_eff == 0:
        p = 1.0
    else:
        k = min(wins, losses)
        tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
        p = min(1.0, 2.0 * tail)
    return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
            "n_effective": float(n_eff), "p_value": p}


def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
                    confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
                    eps: float = 0.0) -> Dict[str, Any]:
    """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).

    ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned by
    fold (same video order). A promotion is honest when the delta CI clears 0 *and* the sign
    test agrees ? not when a single mean ticked up.
    """
    n = min(len(a_values), len(b_values))
    deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
    summary = mean_with_ci(deltas, confidence, n_boot, seed)
    return {
        "mean_delta": summary["mean"],
        "ci_low": summary["ci_low"],
        "ci_high": summary["ci_high"],
        "n": float(n),
        "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
        "sign_test": paired_sign_test(deltas, eps),
    }


def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
    """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest unit.

    The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
    first-seen (deterministic). Groups with no complement (a single group) yield no fold.
    """
    seen: List[Any] = []
    for g in groups:
        if g not in seen:
            seen.append(g)
    n = len(groups)
    folds: List[Tuple[List[int], List[int]]] = []
    for g in seen:
        test = [i for i in range(n) if groups[i] == g]
        train = [i for i in range(n) if groups[i] != g]
        if test and train:
            folds.append((train, test))
    return folds


def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) -> List[Optional[Any]]:
    """Out-of-fold predictions for honest stacking (C3).

    For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on the
    other groups and returns predictions for ``test_idx`` (in that order); results are
```

```
        scattered back to a list aligned to the input items. So a *learned* producer's outputs,
        when used as arbiter features, never include in-sample predictions ? the leakage path
        that inflates measured lift on a tiny corpus. Items in a singleton group stay ``None``
        (needs >= 2 groups to predict).
        """
        n = len(groups)
        oof: List[Optional[Any]] = [None] * n
        for train_idx, test_idx in grouped_folds(groups):
            preds = fit_predict(train_idx, test_idx)
            for pos, ti in enumerate(test_idx):
                if pos < len(preds):
                    oof[ti] = preds[pos]
        return oof

```

14. assistant (2026-06-14T09:50:00.738Z)

Audio columns already exist in the JSONs ? no `--augment` needed. Now let me confirm all 6 JSONs have audio, check the manifest, existing baselines, and QUALITY_ITERATIONS for the known numbers.

15. user (2026-06-14T09:50:03.902Z)

```
adarsh_avi_singles           n_cand=172 n_pos= 80 n_gts=96 audio=True fps=59.94
gbaaddy                      n_cand= 82 n_pos= 15 n_gts=48 audio=True fps=59.94
kushagra_singles             n_cand= 20 n_pos=  4 n_gts=32 audio=True fps=59.94
largetest_doubles            n_cand= 86 n_pos= 40 n_gts=49 audio=True fps=29.97
mahadevpura_singles          n_cand= 57 n_pos= 20 n_gts=31 audio=True fps=29.97
testlarge_short              n_cand= 14 n_pos=  3 n_gts= 6 audio=True fps=29.97
---MANIFEST---
```


```
[
  {
    "name": "adarsh_avi_singles",
    "trajectory": "C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/adarsh_avi_full_traj.csv",
    "fps": 59.94,
    "frame_width": 1920,
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/Adarsh and Avi.rallies.csv"
  },
  {
    "name": "kushagra_singles",
    "trajectory": "C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/kushagra_traj.csv",
    "fps": 59.94,
    "frame_width": 1920,
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/KushagraYashAviFirstVideo.rallies.csv"
  },
  {
    "name": "largetest_doubles",
    "trajectory": "C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/largetestvideo_traj.csv",
    "fps": 29.97,
    "frame_width": 1920,
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/LargeTestVideo.rallies.csv"
  },
  {
    "name": "gbaaddy",
    "trajectory": "C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/gbaaddy_traj.csv",
```

    "fps": 59.94,
    "frame_width": 1920,
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/gBaaddy20May2026.rallies.csv"
  },
  {
    "name": "testlarge_short",
    "trajectory": "C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/testlarge_traj.csv",
    "fps": 29.97,
    "frame_width": 1920,
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/TestLargeVideo.rallies.csv"
  },
  {
    "name": "mahadevpura_singles",
    "trajectory": "C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/mahadevpura_traj.csv",
    "fps": 29.97,
    "frame_width": 1920,
    "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/MahadevpuraSingles.rallies.csv"
  }
]

16. user (2026-06-14T09:50:05.327Z)

```
experiment_leaderboard.json
gemini_wrapper_2026-06-10.json
heuristic_lovo_n6.json
---LEADERBOARD---
rows: 2
```

{
 "timestamp": "2026-06-14T06:23:51.507393+00:00",
 "iou": 0.5,
 "label_set_hash": "50d98ffbc370",
 "num_videos": 6,
 "variant_a": {
  "name": "shuttle_only",
  "spec_hash": "47c867c36f2f",
  "aggregate": {
   "precision": 0.31807603780927995,
   "recall": 0.3101947096042718,
   "f1": 0.3068611057861412,
   "mean_iou": 0.5791831889148886
  }
 },
 "variant_b": {
  "name": "shuttle_person_fusion",
  "spec_hash": "253f4709b4c4",
  "aggregate": {
   "precision": 0.3617228001510626,
   "recall": 0.2708939086387243,
   "f1": 0.2863398846047612,
   "mean_iou": 0.5690111885714347
  }
 },
 "delta": {
  "precision": 0.04364676234178266,
  "recall": -0.0393008009655475,
  "f1": -0.02052122118138,
  "mean_iou": -0.01017200034345389
 },
 "honest_delta_f1": {
  "mean_delta": -0.02052122118137999,

```
  "ci_low": -0.16462806145542527,
  "ci_high": 0.16224446866060574,
  "n": 6.0,
  "ci_excludes_zero": false,
  "sign_test": {
   "wins": 2.0,
   "losses": 4.0,
   "ties": 0.0,
   "n_effective": 6.0,
   "p_value": 0.6875
  }
 }
}
{
 "timestamp": "2026-06-14T06:24:56.487711+00:00",
 "iou": 0.5,
 "label_set_hash": "50d98ffbc370",
 "num_videos": 6,
 "variant_a": {
  "name": "shuttle_person",
  "spec_hash": "ee89f1e41604",
  "aggregate": {
   "precision": 0.3617228001510626,
   "recall": 0.2708939086387243,
   "f1": 0.2863398846047612,
   "mean_iou": 0.5690111885714347
  }
 },
 "variant_b": {
  "name": "shuttle_person_audio",
  "spec_hash": "d68c39eba87c",
  "aggregate": {
   "precision": 0.4530798177857001,
   "recall": 0.3408138120839734,
   "f1": 0.3655826552797436,
   "mean_iou": 0.7074802852140344
  }
 },
 "delta": {
  "precision": 0.0913570176346375,
  "recall": 0.06991990344524907,
  "f1": 0.07924277067498242,
  "mean_iou": 0.13846909664259965
 },
 "honest_delta_f1": {
  "mean_delta": 0.07924277067498244,
  "ci_low": -0.053508315003560535,
  "ci_high": 0.2029045239922802,
  "n": 6.0,
  "ci_excludes_zero": false,
  "sign_test": {
   "wins": 4.0,
   "losses": 2.0,
   "ties": 0.0,
   "n_effective": 6.0,
   "p_value": 0.6875
  }
 }
}
```


17. assistant (2026-06-14T09:50:09.187Z)

All 6 JSONs have audio columns ? no `--augment` needed. The leaderboard already has P3 (person) and P4 (incremental audio). Now let me read the existing baselines and the QUALITY_ITERATIONS doc for the known Gate-A numbers and the heuristic/Gen-0/wrapper baselines.

18. user (2026-06-14T09:50:11.977Z)

=== heuristic_lovo_n6.json ===
{
 "name": "heuristic_velocity_windowing_LOVO",
 "description": "The honest cross-video floor the Gen-0 learned model must beat: calibrated velocity-windowing heuristic, leave-one-video-out over the n=6 owned golden corpus.",
 "computed": "2026-06-10, reproduced via: python -m backend.eval.calibrate_local --manifest output/human_lovo_manifest.json --metric f1 (matches docs/archives/quality-iterations/2026-06-multivideo-lovo)",
 "mean_f1": 0.48,
 "std": 0.242,
 "per_video": {
  "adarsh_avi_singles": 0.751,
  "kushagra_singles": 0.157,
  "largetest_doubles": 0.737,
  "gbaaddy": 0.309,
  "testlarge_short": 0.267,
  "mahadevpura_singles": 0.657
 }
}

=== gemini_wrapper_2026-06-10.json ===
{
 "name": "gemini_wrapper_baseline",
 "description": "Commodity-VLM wrapper baseline: the full pipeline's `gemini` segmenter scored vs human golden labels at IoU 0.5. The commoditization/moat reference the owned model must be judged against \u2014 on multi-court footage the wrapper drops 0.93 -> 0.66, so the owned model's target is the multi-court number, not the clean-footage one.",
 "model": "gemini-flash-latest (resolved 2026-06-10; 3.5-flash era)",
 "conditions": "serial uploads (ai_max_workers=1), chunks re-encoded to 1280px w/ audio (ai_chunk_scale_width), 6-attempt exponential generate retry; min_segment_duration=3.0 filter active (rejects real ~2.5s rallies \u2014 see notes)",
 "per_video": {
  "mahadevpura_singles": {
   "f1": 0.93,
   "precision": 1.0,
   "recall": 0.87,
   "mean_iou": 0.8,
   "boundary_err_s": 0.7,
   "n_gt": 31,
   "n_pred": 27,
   "notes": "3 of 4 FNs were detected by Gemini but rejected by min_segment_duration=3.0 (2.5s rallies). Chunk 6 (unlabeled tail) lost to 503s \u2014 no effect on the labeled window."
  },
  "kushagra_singles": {
   "f1": 0.66,
   "precision": 0.69,
   "recall": 0.62,
   "mean_iou": 0.73,
   "boundary_err_s": 1.2,
   "n_gt": 32,
   "n_pred": 29,
   "notes": "Multi-court (contrast 1.4x). Full 7/7 chunk coverage, audio intact. FPs cluster as background-game pickups + one 53s dead-time merge \u2014 the exact confound the contrast metric predicts."
  }
 },
 "comparison_at_time_of_run": {
  "heuristic_lovo": {
   "mahadevpura_singles": 0.657,
   "kushagra_singles": 0.157
  },
  "gen0_learned_lovo": {
   "mahadevpura_singles": 0.744,

```
   "kushagra_singles": 0.561
  }
 },
 "reliability_note": "gemini-flash-latest served 503 capacity waves throughout (one chunk needed a ~5min backoff); a sole-provider dependency is operationally flaky \u2014 the provider-registry fallback design is load-bearing.",
 "cost_note": "well under $1 for both videos at flash pricing (serial, ~14 chunk requests total)."
}
```


19. user (2026-06-14T09:50:12.798Z)

```
eval_baselines/experiment_leaderboard.json
eval_baselines/gemini_wrapper_2026-06-10.json
eval_baselines/heuristic_lovo_n6.json
---
docs/QUALITY_ITERATIONS.md
212 docs/QUALITY_ITERATIONS.md
```

20. user (2026-06-14T09:50:15.304Z)

```
# Rally-detection quality iterations ? living log

**Purpose.** A running, reverse-chronological log of how we evolve rally-detection quality: each
iteration's hypothesis, what shipped, the measured (or expected) effect, and the lesson. This is the
**live** continuation of the historical trail in
[`archives/quality-iterations/`](archives/quality-iterations/README.md) ? per the archive policy
(`CLAUDE.md`), **terminal-state** (DONE/REJECTED) work gets a dated subdir there; this file stays at
`docs/` top level and keeps growing, then its entries graduate to the archive when they conclude.

**Methodology (non-negotiable).** Every quality change rides the experiment harness
([EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md)): new cues land **flag-gated, default OFF**; they're
graded **offline against golden labels** (per-video + LOVO-honest); promotion to default happens **only
on measured lift** through the `run_regression.py` gate; **rollback = flip a config flag, never
`git revert`.** Cues are designed in [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md); scoring
in [EVALUATION.md](EVALUATION.md); the 2-layer model in
[HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md).

**Golden corpus = the 6 labelled matches** in `output/human_lovo_manifest.json` (owner-confirmed
2026-06-13: use all 6 ? `adarsh_avi_singles, kushagra_singles, largetest_doubles, gbaaddy,
testlarge_short, mahadevpura_singles`). The model trained on these is what we run alongside the existing
detector on NEW footage for the side-by-side review.

---

## Baselines that drive strategy (IoU 0.5 vs human golden labels)

| Path | Mahadevpura (clean-ish) | Kushagra (multi-court) |
|---|---|---|
| Heuristic (velocity windowing) | 0.657 | 0.157 |
| Gen-0 owned head (LOVO, n=6) | 0.744 | 0.561 |
| Two-stage WASB+Gemini refine | 0.83 | ? |
| Gemini wrapper (`gemini-flash-latest`) | 0.93 | 0.66 |
```

Reading: clean footage is commoditized (serve via Gemini); **multi-court drops the commodity wrapper to 0.66 ? that is the owned model's ship target.** Baselines tracked in `eval_baselines/`. The owner's observed false positive ? a player **holding/flicking** the shuttle between points counted as a rally ? is the precision bug the fusion gates target.

Current track (live) ? newest first

Fusion P3 + P4 MEASURED on the n=6 golden corpus ? person = no lift, audio = promising (not promotable) *(2026-06-14)*
The GPU person-feature extraction finished all 6 golden videos (`fusion_golden.py` ? `output/*_golden_features.json`, the replayable substrate); the CPU LOVO drivers then measured the lift with **honest stats (C1 ? bootstrap 95% CI + paired sign test; never a bare mean at n=6)**:

Variant (honest LOVO, IoU ? 0.5)	F1	?F1 vs prior	95% CI	sign test	CI clears 0
shuttle-only	0.307	?	?	?	?
+ person (P3)	0.286	**?0.021** (?6.7%)	[?0.165, +0.162]	2W/4L (p=0.688)	**No**
+ person + audio (P4 incr.)	0.366	**+0.079** (+27.7%)	[?0.054, +0.203]	4W/2L (p=0.688)	**No**

Read honestly: neither cue clears the promotion bar (both CIs span 0, both sign tests n.s.) at n=6 ? **both stay default-OFF** (promote-only-on-lift). **Person** adds noise (no court polygons ? `players_in_court` counts spectators); per-video it's a wash (testlarge +0.40, gbaaddy ?0.22). **Audio** is the more promising lead ? +0.079 / +27.7%, wins 4/6 (largetest +0.27, mahadevpura +0.27), and shuttle+person+audio (0.366) lands **above** shuttle-only (0.307) ? but n=6 can't confirm it. **Next:** more matches (raise N until the sign test can resolve ~+0.08), and measure audio on shuttle-only *directly* (without the person handicap). Recorded in `eval_baselines/experiment_leaderboard.json`. **Negative/uncertain results are kept on purpose ? this is the honest ablation backbone for the paper aim.**

Acting on the first A/B: Gate-A PROMOTED + learned variant FIXED + GX010125 corroboration *(2026-06-14)*
Gate-A (net-crossings ? 2) ? PROMOTED (first cue promoted via the harness). The measured win (?F1 +0.074, **6/6 golden matches, sign-test p=0.031**, CI [+0.042,+0.104]; over-seg 1.68?1.01 ? the [first-A/B archive](archives/quality-iterations/2026-06-first-ab-experiment/README.md)) clears the honest bar. **Recommended config:** `indexing.fusion.min_crossings = 2` on the fusion path (the eval-only Gate A folded into production, MULTI_SIGNAL_FUSION_PLAN §3.1). ? The offline win is on the recall-oriented *proposal* candidate set; flipping the **live served default** stays owner-gated until a real fusion-path run confirms it (the production fusion segmenter is P2/unverified on real video) ? so this records the promotion + rationale, not a silent default flip. **Corroborated on real footage:** offline re-score of GX010125_1080p's *cached* trajectory (no GPU, isolated `output/GX010125_run/`) ? 45 candidates ? CONTROL 45 rallies vs **Gate-A 27**; Gate-A dropped **18 low-crossing windows (~1.7 min)** of likely held-shuttle/feed FPs

(no golden labels for this footage ? divergence, not F1).

Learned variant FIXED ? [#130](https://github.com/avidullu/badminton-highlight-indexer/pull/130). The first A/B's learned logistic lost to CONTROL and zeroed `testlarge` ? a fixed-0.5-cutoff artifact. Added additive/default-OFF `Variant.tune_threshold` (operating point chosen on the **train** fold) + `calibrate` (Platt/isotonic, C2). Validated on the n=6 golden set: **mean F1 0.307 ? 0.423 (+0.116)**, testlarge 0.000 ? 0.300, and the learned variant now **beats CONTROL (0.388)**. Threshold-in-LOVO is the big lever; calibration is available too.

Next: a real fusion-path run to confirm Gate-A served-side; per-video output layout (PER_VIDEO_OUTPUT_LAYOUT.md) so each video's artifacts are self-contained.

First end-to-end A/B on the golden corpus (n=6) ? **ARCHIVED** *(2026-06-14)*
First real use of the harness + C1?C4 helpers, 100% offline (re-scored persisted WASB trajectories, no GPU). **Gate-A (net-crossings ? 2) is a measured win** ? ?F1 +0.074, 6/6 matches, sign-test **p = 0.031**, CI [+0.042, +0.104], over-segmentation (segment-count ratio) **1.68 ? 1.01**. The window-level 5-feature logistic **did not clear the bar** (?0.081, 2W/4L; a threshold/calibration artifact ? *not* the stronger per-frame prototype). The framework held up: **C1 promoted one variant and refused another; C4 surfaced over-segmentation tIoU hid.** Full record + tables ? [`archives/quality-iterations/2026-06-first-ab-experiment/`](archives/quality-iterations/2026-06-first-ab-experiment/README.md). Next: Gate-A is a candidate for the no-regression promotion gate; the logistic failure is the C2 (calibration) + threshold-in-loop task list.

Eval-honesty helpers (course corrections C1?C4) ? additive, default-OFF *(2026-06-14)*
Why. A multi-agent literature review (folded into ANALYZERS_AND_RECONCILERS.md §13) confirmed the framework's *skeleton* is standard prior art (Temporal Action Localization/Segmentation + late/score-level fusion ? cite, don't claim), and flagged that our **measurement** is walking into known small-N traps that would make "promote only on measured lift" promote noise or hide a real win. Four corrections, shipped as **pure, additive helpers** (no existing `compare`/LOVO/`metrics.score` behaviour changed ? callers opt in):
- **C1 ? never a single mean.** `backend/eval/eval_stats.py`: `mean_with_ci`/`bootstrap_ci` (deterministic, seeded), `paired_sign_test` (the exact distribution-free "won on enough *videos*?" test the corpus already needs), `paired_delta_ci` (B?A with a CI that must clear 0). At n?6 a lone LOVO mean is not trustworthy.
- **C2 ? calibrate cue confidence.** `backend/eval/score_calibration.py`: Platt + isotonic (PAVA) calibrators + `brier_score`/`expected_calibration_error`, so a CNN softmax, an audio onset, and an integer net-crossing count are comparable before the arbiter and the over-confident cue can't swamp the rest.
- **C3 ? honest stacking.** `eval_stats.out_of_fold_predictions`/`grouped_folds` (leave-one-group-out by match) so a *learned* producer's outputs used as arbiter features are out-of-fold (no leakage).

- **C4 ? over-segmentation-visible metrics.** `backend/eval/segmentation_metrics.py`: `f1_at_overlaps` (F1@{10,25,50}), `average_precision`/`mean_average_precision` (mAP@tIoU), `segment_count_ratio`. tIoU alone is blind to a rally split in 3 or two rallies merged ? exactly what the reconciler exists to fix.

Status. Helpers + 24 unit tests landed (pure stdlib, run in the numpy/GPU-free lanes); ruff + mypy clean. **Not yet wired into the default harness flow** (wiring `compare`/`run_regression.py` to report CIs + F1@k is the follow-on, owner-gated). The discipline they encode is the methodology for every future cue measurement ? see §13/C1?C4 for the citations and the remaining (Tier-2/3) corrections.

Multi-signal fusion P2: the FusionSegmenter *(in progress, 2026-06-14)*
Hypothesis. Adjudicating shuttle-seeded windows with the net-crossing **Gate A** (`min_crossings?2`) and a person-occupancy **Gate B** raises **precision** on the held-shuttle / warm-up / one-sided-feed / spectator false positives ? *without* hurting recall, because the shuttle stays the primary, camera-invariant seed.

Scope ? and an honesty boundary from the archive (read this). The [`2026-06-multivideo-lovo`](archives/quality-iterations/2026-06-multivideo-lovo/README.md) iteration already smoke-tested **court-region detection as a fix for the *multi-court contrast* confound** and found it **low-headroom**: only 12?14% of dead-time shuttle detections fall *outside* the foreground court, so a perfect court gate lifts contrast to only ~1.6?2.2× (below the ?3.6× "works" threshold) ? the multi-court problem is **temporal**, and the **learned model**, not a court mask, is its answer. So Gate B here is **NOT** sold as the multi-court fix. It targets a **different, previously-untested** failure mode (held-shuttle/warm-up/one-sided/spectator FPs). Two distinct levers; don't conflate them.

Data support for Gate A. The same archive shows real rallies on the multi-court matches contain **?2 net-crossings 97% of the time (median 5)** while a single service feed crosses ~once ? `min_crossings?2` is a principled, data-backed discriminator (it's the held-shuttle owner-bug killer).

Shipping shape. A NEW `fusion` segmenter, **default OFF** (`default_segmenter` unchanged); Gate A wired into the live trajectory path behind `indexing.min_crossings` (default 0 = OFF ? wasb/tracknet byte-identical today); the court mask is an **optional** config polygon (omit ? Gate B is player-count-only; supply ? masked). Person features (`players_in_court, two_sided_motion, mean_player_motion, in_court_ratio, shuttle_player_colocation`) persisted into candidate `stats` for P3's learned fusion. Reuses `PersonDetector` (P1) + `rally_gate` (already existed) ? no reinvention.

Status (2026-06-14): SHIPPED, default-OFF. `FusionSegmenter` (`backend/pipeline/segmenters/fusion_hybrid.py`, registry key `fusion_hybrid`) + the pure feature math (`backend/pipeline/detectors/fusion_features.py`) + two identity hooks on the trajectory engine (`_enrich_candidate_stats`, `_select_for_handoff`) + `PersonDetector.

detections_per_frame` + the `FusionConfig` block. Built design-first (a `fusion-p2-design` fan-out workflow), then
adversarially reviewed by a 31-agent workflow (25 findings ? 19 confirmed): the headline confirmation is
NO-REGRESSION verified ? wasb/tracknet twins persist byte-identical stats and hand off all windows (pinned by
the extended equivalence test); default `FusionSegmenter` == its wasb substrate. The confirmed low/medium findings
were fixed (frame-dim guards, `round()` frame indices, the per-window axis-estimate redundancy ? a reusable
`rally_gate` estimate, honest torch-import docstrings, and 6 new tests closing the person-detector / Gate-B /
substrate / colocation-halo gaps). Suite 441?**467 green**; ruff + mypy clean.
Two findings deferred with intent (reviewer-agreed): person features are computed eagerly even for
Gate-A-pruned windows (a P3 redesign, would change the persisted-superset semantics); and `fusion.velocity_threshold`
must be set equal to the substrate's to A/B against a tuned wasb run (documented in `FusionConfig`).
Lift still to be measured offline on the 6 golden videos via `experiment.compare(CONTROL, fusion)`; the
held-shuttle-drop + person-feature values are GPU/owner-side. *(Measured numbers appended here when run ? no claims before.)*

Person cue extracted to a reusable producer (fusion P1) ? [#115](https://github.com/avidullu/badminton-highlight-indexer/pull/115), 2026-06-13
Lifted the torchvision detector + ByteTrack + velocity windowing out of `yolo_hybrid` into
`backend/pipeline/detectors/person_detector.py::PersonDetector`. **No behavior change** (proven by the
unchanged `process_video` output tests); `yolo_hybrid` now orchestrates only. **Why it's a quality move:**
it makes "add the person cue to fusion" an *add-a-producer*, not a segmenter rewrite. Suite 440?441 green.

Experiment harness: A/B without rollback risk ? [#114](https://github.com/avidullu/badminton-highlight-indexer/pull/114), 2026-06-13
Shipped `backend/eval/experiment.py` ? the thin variant/experiment layer (the audit's "~80% already
exists" held; pure glue over `metrics`/`calibration`/`classifier`/`training`/`regression.gt_hash`). Gives
`Variant` (+ pinned `CONTROL`), `compare()` (labeled A/B, LOVO-honest), **`compare_unlabeled()`** (the SxS
read on NEW unlabeled footage ? agreed / A-only / B-only), and a JSON experiment leaderboard. **This is
the instrument every iteration is measured on.** 19 tests incl. the no-regression invariant (all cues OFF
? byte-identical to control). Suite 421?440 green.

Archived predecessors (the historical trail)
Completed iterations live under [`archives/quality-iterations/`](archives/quality-iterations/README.md):
- **`2026-06-initial/`** ? seed of the quality phase.
- **`2026-06-human-gt-tuning/`** ? first eval vs **human** GT on a partial-label video; partial-label
  harness + labeled-window methodology; baseline 0.650 ? tuned 0.742 (*fit-on-self*); found+fixed a
  negative-start ingestion bug (collector PR #13).
- **`2026-06-multivideo-lovo/`** ? the **n=6 corpus + honest LOVO**; established the heuristic is
  footage-fragile (multi-court Kushagra 0.16 / gBaaddy 0.28), the **"contrast" metric** predicts
  heuristic F1 monotonically, and a **learned per-frame prototype overtook the heuristic

at n=3 and holds
187 ~+0.10** (n=6: learned 0.583 vs heuristic 0.480), with **AUC 0.83?0.92 decoupled from contrast**. This
188 is the data-backed green light for the owned model.
189
190 ---
191
192 ## Standing lessons (carry-forward)
193 - **Persist detection once, re-score variants forever.** Expensive/risky = per-frame detection (GPU/WSL);
194 cheap/swappable = the decision. Most experiments never enter the served pipeline.
195 - **LOVO or it didn't happen.** Score on a held-out *video*, never a held-out window from the same video.
196 Group by **match**, not file. Sign test needs n ? ~n=10/9-wins for p<0.05.
197 - **The multi-court confound is TEMPORAL, not spatial/loudness** ? three heuristics were tested and failed
198 for the same reason (foreground spatial gate **?0.04**; audio energy **AUC ~0.6**; court-region
199 separability **12?14% out-of-region**): the dead-time confusion is rally-vs-non-rally activity in the
200 *same place and frequency band*. Only the learned model's temporal/structural features crack it (AUC
201 0.83 on Kushagra/gBaaddy). **Don't re-derive these dead ends.**
202 - **Court-masking ? multi-court fix** (see above) ? it's a **precision** tool for off-court persons and
203 one-sided feeds, a *different* lever; measure its lift, don't assume it.
204 - **Net-crossings are a strong rally signal** ? 97% of real rallies cross ?2× (median 5); single feeds ~1×.
205 Gate A leans on this.
206 - **Small-N feature discipline.** Few, scale/translation-invariant features (counts/ratios, not raw
207 coords) + a logistic regression, not a net ? raw coords memorize this court/camera.
208 - **Calibration, not representation, is the small-N wall** ? learned probabilities separate (high AUC) but
209 the operating point doesn't transfer with few training videos. More distinct matches is the highest-ROI
210 unblock.
211 - **Gate A already existed, just wasn't wired to production** ? the cheapest precision win is plumbing, not
212 new capability. Always check what the eval drivers already do that the live path doesn't.
213

21. assistant (2026-06-14T09:50:18.460Z)

I now have full context. Let me look at the first-A/B archive for the exact Gate-A numbers and over-seg ratio, and check the `distill_local` feature names. Then I'll build and run the leave-one-signal-out ablation.

22. user (2026-06-14T09:50:23.158Z)

README.md

FEATURE_NAMES: ['duration', 'peak_velocity', 'mean_velocity', 'active_ratio', 'net_crossings']
---audio---
AUDIO: ['audio_hit_count', 'audio_hit_rate', 'audio_hit_strength_mean']
---person---
PERSON: ['players_in_court', 'two_sided_motion', 'mean_player_motion', 'in_court_ratio', 'shuttle_player_colocation']

23. user (2026-06-14T09:50:31.284Z)

1 # First end-to-end A/B experiment on the golden corpus (n=6)
2

STATUS: COMPLETED (2026-06-14). The first time the experiment harness ([EXPERIMENT_HARNESS.md](../../../EXPERIMENT_HARNESS.md)) + the measurement-honesty helpers (course corrections C1?C4, [ANALYZERS_AND_RECONCILERS.md](../../../ANALYZERS_AND_RECONCILERS.md) §13) were run end-to-end on real golden data ? a posterity record of the framework's first real use.

Trigger. After landing the signal-fusion framework doc + §13 literature review and the additive C1?C4 helpers (`backend/eval/eval_stats.py`, `score_calibration.py`, `segmentation_metrics.py`, PR #124), the question was: *does the framework actually hold up on a golden video?* This is the validation run.

Method (100% offline ? the "persist detection once, re-score forever" path)
- **No GPU.** Re-scored the **persisted WASB shuttle trajectories** of the 6 golden matches (`output/human_lovo_manifest.json`); detection was never re-run.
- **One consistent candidate set per video** via `distill_local.build_candidates` (recall-oriented proposal windowing `velocity_thresh=0.005, merge_gap=1.0, min_dur=0.5`) ? per-window shuttle features `[duration, peak_velocity, mean_velocity, active_ratio, net_crossings]`; golden labels via `calibrate_wasb.load_golden_gts`; per-candidate y via the evaluator's own IoU matcher.
- **Three variants re-scored on that same set** (`backend/eval/experiment.py`):
  - **CONTROL** ? merge all candidates (no filter).
  - **Gate-A** ? keep candidates with `net_crossings ? 2`, then merge (the held-shuttle / service-feed structural FP killer; `rally_gate`).
  - **learned** ? a window-level logistic regression over the 5 shuttle features, **leave-one-VIDEO-out** (train on the other 5 matches, predict held-out), prob ? 0.5, merge.
- **Graded with:** temporal-IoU F1 (per video), **C1** honest aggregation (`mean_with_ci`, `paired_delta_ci`, exact `paired_sign_test`), **C4** segmentation metrics (`f1_at_overlaps`, `segment_count_ratio`), and a **C3** out-of-fold learned mAP@tIoU.
- IoU 0.5. Repro: `scratch/ab_n6_lovo.py` (machine-side; reads the gitignored manifest + the external golden trajectories/labels); raw result `output/ab_experiment_n6.json`.

Corpus
video	candidates	golden rallies	proposal recall ceiling
adarsh_avi_singles	172	96	0.833
kushagra_singles (multi-court)	20	32	**0.125**
largetest_doubles	86	49	0.816
gbaaddy (multi-court)	82	48	**0.312**
testlarge_short	14	6	0.500
mahadevpura_singles	57	31	0.645

The multi-court matches (kushagra, gbaaddy) have **low proposal-recall ceilings** ? the decision stage can never recover rallies the windowing missed (the proposal-recall ceiling pitfall, §13). Their absolute F1 is windowing-capped, not a decision failure.

Results

Per-video held-out F1 (IoU 0.5):
video	CONTROL	Gate-A	learned (LOVO)
adarsh_avi_singles	0.597	**0.722**	0.613

kushagra_singles	0.154	**0.163**	0.163
largetest_doubles	0.593	**0.667**	0.535
gbaaddy	0.231	**0.274**	0.222
testlarge_short	0.300	**0.375**	**0.000**
mahadevpura_singles	0.455	**0.571**	0.308
mean	**0.388**	**0.462**	**0.307**

A/B 1 ? CONTROL vs Gate-A (static). ?F1 **+0.074**, 95% CI **[+0.042, +0.104]** (excludes 0),
sign test **6 wins / 0 losses, p = 0.031**. ? **A real, statistically-supported win** ? Gate-A improves F1
on *every* golden match.

A/B 2 ? CONTROL vs window-level learned logistic (LOVO). ?F1 **?0.081**, CI **[?0.173, ?0.002]**,
sign test **2 wins / 4 losses, p = 0.688**. ? **Does not clear the bar** ? this variant is *worse* than
CONTROL on this setup (and the mean is dragged by two big per-video losses; the sign test is inconclusive ?
the two honest views disagreeing is itself a reason to report both).

C4 ? over-segmentation-aware metrics (mean across videos):

variant	F1@10	F1@25	F1@50	segment-count ratio
CONTROL	0.645	0.603	0.388	1.68
Gate-A	**0.748**	**0.698**	**0.462**	**1.01**
learned	0.564	0.509	0.307	0.89

Gate-A drives the **segment-count ratio 1.68 ? 1.01** ? it nearly eliminates the over-segmentation that
plain temporal-IoU barely reflected. The learned variant's 0.89 is *under*-segmentation (it drops real
rallies). **C3 out-of-fold learned mAP@tIoU[.3?.7] = 0.245** (weak ranking ? corroborates the loss).

Wins / losses / lessons
- ? **Gate-A (net-crossings ? 2) is a genuine win** ? +0.074 mean F1, 6/6 matches, p = 0.031, CI clears 0,
  over-segmentation 1.68?1.01. Promote-worthy by the honest gate. (Confirms the held-shuttle owner-bug
  killer on the whole corpus, offline, no GPU.)
- ? **The window-level 5-feature logistic (threshold 0.5) is not a win here** ? ?0.081, mixed (2W/4L).
  `testlarge_short ? 0.000` (the model predicted *zero* rallies) is a **threshold/calibration artifact**, a
  textbook motivation for **C2 (calibrate cue confidence)** and tuning the threshold *inside* the LOVO loop.
- ?? **Do not conflate this with the documented learned winner.** The
  [`2026-06-multivideo-lovo`](../2026-06-multivideo-lovo/README.md) result (learned 0.583 vs heuristic
  0.480) is a **per-frame prototype** (`rally_seq_proto.py`), a *different, stronger* model than this
  window-level logistic. This experiment compared a *weaker* learned path against a *strong* CONTROL
  (merge-all of a recall-oriented proposal) ? a different matchup. The takeaway is methodological, not
  "learned fusion fails."
- ? **The methodology itself worked.** C1 gave a confident **YES** on Gate-A *and* a clear **NO** on the
  logistic ? preventing a wrong promotion in **both** directions. A bare-mean report would have hidden
  Gate-A's significance and might have over-read the logistic's mean. C4 surfaced the over-segmentation
  story tIoU buried. This is exactly the framework holding up.
- ? **Multi-court is windowing-capped** ? kushagra/gbaaddy absolute F1 is bounded by the

0.125/0.312
proposal-recall ceiling; the decision layer can't fix what the proposal missed (measure AR@AN separately).

Significance
First real A/B on the golden corpus ? the harness + C1?C4 produced an **honest, promotable finding (Gate-A)** and an **honest rejection (this logistic)**, entirely offline. This is the empirical backbone the paper aim ([`paper-coauthor-aim`]) needs ? honest measured results only. Gate-A becomes a candidate for the no-regression promotion gate; the logistic's failure mode is the concrete task list for C2 + threshold-in-loop.

Links: [EXPERIMENT_HARNESS.md](../../../EXPERIMENT_HARNESS.md) · [ANALYZERS_AND_RECONCILERS.md](../../../ANALYZERS_AND_RECONCILERS.md) §13 (C1?C4) · [QUALITY_ITERATIONS.md](../../../QUALITY_ITERATIONS.md) · predecessor [`2026-06-multivideo-lovo`](../2026-06-multivideo-lovo/README.md).

24. assistant (2026-06-14T09:50:38.181Z)

I have everything I need to design the ablation. Key understanding:

- **Feature columns** (consumed by the learned logistic): shuttle (5: duration, peak_velocity, mean_velocity, active_ratio, net_crossings), person (5), audio (3) = 13 total.
- **Gates** (decision masks, not learned columns): Gate-A = `min_crossings` on `net_crossings`; Gate-B = `min_players` on `players_in_court`.

The owner wants a **leave-one-signal-out** ablation. For the learned-fusion layer, the natural "full set" = all 13 feature columns; leave-one-group-out removes each signal group. For the gates, Gate-A is a re-scoring mask, so I'll measure Gate-A ON vs OFF separately (its known result + reproduction + over-seg ratio). Let me write the ablation script in scratch and run it.

Let me mark this as a new chapter and write the ablation runner.

25. user (2026-06-14T09:50:39.400Z)

Chapter marked: "Gate-A/B leave-one-signal-out ablation". Continue your current work.

26. assistant (2026-06-14T09:51:39.402Z)

```
"""Leave-one-signal-out ablation for the Gate-A/B foundational decision (CPU, offline).

Reuses ONLY the shipped harness (experiment.compare + eval_stats) over the 6 persisted
golden feature JSONs (output/*_golden_features.json). No new scoring math, no GPU, no Gemini.

Two kinds of question, two mechanisms (mirrors the repo's own distinction):

1. LEARNED-FUSION marginal contribution (leave-one-signal-out): the "full" learned variant
   uses ALL 13 feature columns (shuttle 5 + person 5 + audio 3). For each signal GROUP (and
   each individual shuttle feature where sensible) we drop those columns and measure held-out
   ?F1 = full - leave_one_out, with bootstrap 95% CI + paired sign test. ?F1 > 0 means the
   signal carries marginal information the learned scorer used.

   NOTE the sign convention: we report delta = FULL - DROPPED, so a POSITIVE delta means the
   signal HELPS (dropping it hurts). experiment.compare reports B - A, so we set
   variant_a = leave_one_out (dropped), variant_b = full  -> B-A = full - dropped. Good.

2. GATE marginal contribution (decision masks, NOT learned columns): Gate-A (min_crossings>=2)
   and Gate-B (min_players>=N) are re-scoring masks over the recall-oriented CONTROL proposal
   set. Measured as static-variant A/B: CONTROL vs CONTROL+gate. Reports F1, ?F1 (CI+sign),
   and the segment-count / over-seg ratio (C4) that tIoU hides.
```

```
All numbers printed are ACTUAL measured values. Writes a JSON blob to scratch/ for the doc.
"""
from __future__ import annotations

import json
import os
from typing import Any, Dict, List, Sequence

from backend.eval import experiment
from backend.eval.eval_stats import paired_delta_ci
from backend.eval import segmentation_metrics as segm
from backend.eval.metrics import score
from backend.pipeline.detectors import audio_features as af
from backend.pipeline.detectors import fusion_features as ff
from backend.eval import distill_local

SHUTTLE = list(distill_local.FEATURE_NAMES)          # 5
PERSON = list(ff.PERSON_FEATURE_NAMES)               # 5
AUDIO = list(af.AUDIO_FEATURE_NAMES)                 # 3
ALL_FEATURES = SHUTTLE + PERSON + AUDIO              # 13

FEATURES_DIR = "output"
IOU = 0.5
THRESHOLD = 0.5


def load_videos() -> List[experiment.VideoCandidates]:
    import glob
    videos: List[experiment.VideoCandidates] = []
    for path in sorted(glob.glob(os.path.join(FEATURES_DIR, "*_golden_features.json"))):
        with open(path, encoding="utf-8") as f:
            d = json.load(f)
        cands = d["candidates"]
        intervals = [(float(c["start"]), float(c["end"])) for c in cands]
        feats: Dict[str, List[float]] = {}
        for fn in SHUTTLE:
            feats[fn] = [float(c["shuttle"][fn]) for c in cands]
        for fn in PERSON:
            feats[fn] = [float(c["person"][fn]) for c in cands]
        for fn in AUDIO:
            feats[fn] = [float(c["audio"][fn]) for c in cands]
        gts = [(float(a), float(b)) for a, b in d["gts"]]
        videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
                                                 features=feats, gts=gts))
    return videos


def paired_from_compare(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
    """Pair B-A per-video held-out scores -> mean delta + CI + sign test."""
    by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
    by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
    vids = [p["video"] for p in res["variant_a"]["per_video"]]
    return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])


def learned_lovo(videos, feature_names) -> Dict[str, Any]:
    """Run a learned LOVO variant over `feature_names`; return aggregate + per-video f1."""
    v = experiment.Variant(name="v", feature_names=list(feature_names), threshold=THRESHOLD)
    ev = experiment.evaluate_variant(videos, v, iou=IOU, honest=True)
    return ev


def leave_one_out_row(videos, full_features, drop, label) -> Dict[str, Any]:
    """full (all features) as B; leave-one-out (full minus `drop`) as A.
    delta = B - A = full - dropped  -> positive means the dropped signal HELPS."""
```

```python
    kept = [f for f in full_features if f not in set(drop)]
    a = experiment.Variant(name=f"drop_{label}", feature_names=kept, threshold=THRESHOLD)
    b = experiment.Variant(name="full", feature_names=list(full_features), threshold=THRESHOLD)
    res = experiment.compare(videos, a, b, iou=IOU, honest=True)  # B-A = full - dropped
    pd = paired_from_compare(res)
    st = pd["sign_test"]
    return {
        "signal": label,
        "dropped_cols": list(drop),
        "f1_full": res["variant_b"]["aggregate"]["f1"],
        "f1_dropped": res["variant_a"]["aggregate"]["f1"],
        "marginal_dF1": pd["mean_delta"],          # full - dropped
        "ci_low": pd["ci_low"], "ci_high": pd["ci_high"],
        "ci_excludes_zero": pd["ci_excludes_zero"],
        "wins": int(st["wins"]), "losses": int(st["losses"]),
        "p_value": st["p_value"],
        "per_video_full": {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]},
        "per_video_dropped": {p["video"]: p["f1"] for p in res["variant_a"]["per_video"]},
    }


def static_predict_segcount(variant, videos) -> Dict[str, float]:
    """Mean F1@k + segment_count_ratio for a static variant across videos."""
    overlaps = segm.DEFAULT_OVERLAPS
    f1k = {ov: [] for ov in overlaps}
    ratios = []
    for vc in videos:
        preds = experiment.predict(variant, vc)
        got = segm.f1_at_overlaps(preds, vc.gts or [], overlaps)
        for ov in overlaps:
            f1k[ov].append(got[ov])
        ratios.append(segm.segment_count_ratio(preds, vc.gts or []))
    out = {f"f1@{int(ov*100)}": (sum(v)/len(v) if v else 0.0) for ov, v in f1k.items()}
    out["segment_count_ratio"] = sum(ratios)/len(ratios) if ratios else 0.0
    return out


def gate_ab(videos, variant_a, variant_b, label) -> Dict[str, Any]:
    """Static A/B (no learned filter): CONTROL vs CONTROL+gate. Reports F1, ?F1 (CI+sign),
    and over-seg ratio for both."""
    res = experiment.compare(videos, variant_a, variant_b, iou=IOU, honest=True)
    pd = paired_from_compare(res)
    st = pd["sign_test"]
    seg_a = static_predict_segcount(variant_a, videos)
    seg_b = static_predict_segcount(variant_b, videos)
    # count of predicted segments per variant (for over-seg narrative)
    def total_preds(v):
        return sum(len(experiment.predict(v, vc)) for vc in videos)
    return {
        "label": label,
        "f1_a": res["variant_a"]["aggregate"]["f1"],
        "f1_b": res["variant_b"]["aggregate"]["f1"],
        "p_a": res["variant_a"]["aggregate"]["precision"],
        "p_b": res["variant_b"]["aggregate"]["precision"],
        "r_a": res["variant_a"]["aggregate"]["recall"],
        "r_b": res["variant_b"]["aggregate"]["recall"],
        "dF1": pd["mean_delta"], "ci_low": pd["ci_low"], "ci_high": pd["ci_high"],
        "ci_excludes_zero": pd["ci_excludes_zero"],
        "wins": int(st["wins"]), "losses": int(st["losses"]), "p_value": st["p_value"],
        "segratio_a": seg_a["segment_count_ratio"], "segratio_b": seg_b["segment_count_ratio"],
        "n_pred_a": total_preds(variant_a), "n_pred_b": total_preds(variant_b),
        "per_video_dF1": {p["video"]: p["f1"] for p in res["per_video_delta"]},
        "f1k_a": seg_a, "f1k_b": seg_b,
    }
```

```
def main() -> None:
    videos = load_videos()
    print(f"Loaded {len(videos)} golden videos: {[v.name for v in videos]}")
    result: Dict[str, Any] = {"n_videos": len(videos), "iou": IOU, "threshold": THRESHOLD}

    # ---- Full learned variant (all 13 cols) reference ----
    full_ev = learned_lovo(videos, ALL_FEATURES)
    result["full_learned"] = {
        "feature_names": ALL_FEATURES,
        "aggregate": full_ev["aggregate"],
        "per_video": {p["video"]: p["f1"] for p in full_ev["per_video"]},
    }
    print("\n=== FULL learned LOVO (all 13 feature columns) ===")
    print(f"  F1={full_ev['aggregate']['f1']:.3f}  P={full_ev['aggregate']['precision']:.3f}  "
          f"R={full_ev['aggregate']['recall']:.3f}")

    # ---- Leave-one-SIGNAL-GROUP-out ----
    print("\n=== Leave-one-signal-out marginal contribution (delta = full - dropped) ===")
    groups = [
        ("person", PERSON),
        ("audio", AUDIO),
        ("net_crossings(shuttle-as-feature)", ["net_crossings"]),
        ("shuttle_velocity_block", ["peak_velocity", "mean_velocity", "active_ratio"]),
        ("duration", ["duration"]),
    ]
    rows = []
    for label, drop in groups:
        row = leave_one_out_row(videos, ALL_FEATURES, drop, label)
        rows.append(row)
        print(f"  drop {label:<36s} ?F1(full-drop)={row['marginal_dF1']:+.3f}  "
              f"CI[{row['ci_low']:+.3f},{row['ci_high']:+.3f}]  "
              f"{row['wins']}W/{row['losses']}L p={row['p_value']:.3f}  "
              f"clears0={row['ci_excludes_zero']}")
    result["leave_one_out"] = rows

    # ---- Also: incremental ADD-each-group-onto-shuttle (matches P3/P4 framing) ----
    print("\n=== Incremental add-onto-shuttle (B - A) ===")
    add_rows = []
    incr = [
        ("shuttle->+person", SHUTTLE, SHUTTLE + PERSON),
        ("shuttle->+audio", SHUTTLE, SHUTTLE + AUDIO),
        ("shuttle+person->+audio", SHUTTLE + PERSON, SHUTTLE + PERSON + AUDIO),
        ("shuttle->+person+audio", SHUTTLE, ALL_FEATURES),
    ]
    for label, a_f, b_f in incr:
        a = experiment.Variant(name="a", feature_names=a_f, threshold=THRESHOLD)
        b = experiment.Variant(name="b", feature_names=b_f, threshold=THRESHOLD)
        res = experiment.compare(videos, a, b, iou=IOU, honest=True)
        pd = paired_from_compare(res)
        st = pd["sign_test"]
        r = {"label": label, "f1_a": res["variant_a"]["aggregate"]["f1"],
             "f1_b": res["variant_b"]["aggregate"]["f1"], "dF1": pd["mean_delta"],
             "ci_low": pd["ci_low"], "ci_high": pd["ci_high"],
             "ci_excludes_zero": pd["ci_excludes_zero"],
             "wins": int(st["wins"]), "losses": int(st["losses"]), "p_value": st["p_value"]}
        add_rows.append(r)
        print(f"  {label:<28s} F1 {r['f1_a']:.3f}->{r['f1_b']:.3f}  ?F1={r['dF1']:+.3f}  "
              f"CI[{r['ci_low']:+.3f},{r['ci_high']:+.3f}]  {r['wins']}W/{r['losses']}L "
              f"p={r['p_value']:.3f} clears0={r['ci_excludes_zero']}")
    result["incremental_add"] = add_rows

    # ---- GATE-A ON vs OFF (static, the foundational decision) ----
    print("\n=== Gate-A ON vs OFF (static CONTROL vs net_crossings>=2) ===")
    control = experiment.Variant(name="control")               # merge-all, no gate
    gate_a = experiment.Variant(name="gate_a", min_crossings=2)
```

```
    ga = gate_ab(videos, control, gate_a, "Gate-A (min_crossings>=2)")
    result["gate_a"] = ga
    print(f"  CONTROL F1={ga['f1_a']:.3f} (P={ga['p_a']:.3f} R={ga['r_a']:.3f})  "
          f"segratio={ga['segratio_a']:.2f}  n_pred={ga['n_pred_a']}")
    print(f"  Gate-A  F1={ga['f1_b']:.3f} (P={ga['p_b']:.3f} R={ga['r_b']:.3f})  "
          f"segratio={ga['segratio_b']:.2f}  n_pred={ga['n_pred_b']}")
    print(f"  ?F1={ga['dF1']:+.3f} CI[{ga['ci_low']:+.3f},{ga['ci_high']:+.3f}] "
          f"{ga['wins']}W/{ga['losses']}L p={ga['p_value']:.3f}
clears0={ga['ci_excludes_zero']}")

    # ---- GATE-B ON vs OFF (static; min_players sweep). No court polygons -> caveat. ----
    print("\n=== Gate-B ON vs OFF (static CONTROL vs players_in_court>=N) ===")
    gb_rows = []
    for n in (1, 2, 3, 4):
        gb = experiment.Variant(name=f"gate_b{n}", min_players=n)
        r = gate_ab(videos, control, gb, f"Gate-B (min_players>={n})")
        gb_rows.append(r)
        print(f"  N>={n}: CONTROL F1={r['f1_a']:.3f} -> Gate-B F1={r['f1_b']:.3f}  "
              f"?F1={r['dF1']:+.3f} CI[{r['ci_low']:+.3f},{r['ci_high']:+.3f}] "
              f"{r['wins']}W/{r['losses']}L p={r['p_value']:.3f} clears0={r['ci_excludes_zero']}
"
              f"n_pred {r['n_pred_a']}->{r['n_pred_b']}")
    result["gate_b"] = gb_rows

    # ---- Gate-A AND Gate-B stacked (does B add on top of A?) ----
    print("\n=== Gate-A + Gate-B stacked vs Gate-A alone ===")
    gab_rows = []
    for n in (1, 2):
        a = experiment.Variant(name="gate_a", min_crossings=2)
        b = experiment.Variant(name=f"gate_ab{n}", min_crossings=2, min_players=n)
        res = experiment.compare(videos, a, b, iou=IOU, honest=True)
        pd = paired_from_compare(res)
        st = pd["sign_test"]
        r = {"n": n, "f1_a": res["variant_a"]["aggregate"]["f1"],
             "f1_b": res["variant_b"]["aggregate"]["f1"], "dF1": pd["mean_delta"],
             "ci_low": pd["ci_low"], "ci_high": pd["ci_high"],
             "ci_excludes_zero": pd["ci_excludes_zero"],
             "wins": int(st["wins"]), "losses": int(st["losses"]), "p_value": st["p_value"]}
        gab_rows.append(r)
        print(f"  GateA + B(N>={n}) vs GateA: F1 {r['f1_a']:.3f}->{r['f1_b']:.3f}
?F1={r['dF1']:+.3f} "
              f"CI[{r['ci_low']:+.3f},{r['ci_high']:+.3f}] {r['wins']}W/{r['losses']}L
p={r['p_value']:.3f}")
    result["gate_ab_stacked"] = gab_rows

    out = os.path.join("scratch", "gate_ab_ablation_result.json")
    with open(out, "w", encoding="utf-8") as f:
        json.dump(result, f, indent=2)
    print(f"\nWrote {out}")


if __name__ == "__main__":
    main()
```

27. user (2026-06-14T09:51:39.420Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\scratch\gate_ab_ablation.py (file state is current in your context ? no need to Read it
back)
```

28. user (2026-06-14T09:51:42.979Z)

```
Exit code 1
Traceback (most recent call last):
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\gate_ab_ablation.py", line
```

31, in <module>
    from backend.eval import experiment
ModuleNotFoundError: No module named 'backend'

29. assistant (2026-06-14T09:51:45.365Z)

Need to run from the repo root with proper module path. The script imports work as a module; let me run with `PYTHONPATH` set to the repo root.

30. user (2026-06-14T09:51:46.640Z)

```
Exit code 1
/usr/bin/bash: line 1: :PYTHONPATH: command not found
Traceback (most recent call last):
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\gate_ab_ablation.py", line
31, in <module>
    from backend.eval import experiment
ModuleNotFoundError: No module named 'backend'
```

31. assistant (2026-06-14T09:51:49.294Z)

The Bash tool is POSIX (Git bash), not PowerShell. Let me use the PowerShell tool for env var handling, or set it inline.

32. user (2026-06-14T09:51:57.382Z)

```
Loaded 6 golden videos: ['adarsh_avi_singles', 'gbaaddy', 'kushagra_singles',
'largetest_doubles', 'mahadevpura_singles', 'testlarge_short']

=== FULL learned LOVO (all 13 feature columns) ===
  F1=0.366  P=0.453  R=0.341

=== Leave-one-signal-out marginal contribution (delta = full - dropped) ===
  drop person                               ?F1(full-drop)=-0.009  CI[-0.074,+0.055]  2W/4L
p=0.688  clears0=False
  drop audio                                ?F1(full-drop)=+0.079  CI[-0.054,+0.203]  4W/2L
p=0.688  clears0=False
  drop net_crossings(shuttle-as-feature)    ?F1(full-drop)=-0.014  CI[-0.083,+0.027]  4W/1L
p=0.375  clears0=False
  drop shuttle_velocity_block               ?F1(full-drop)=+0.002  CI[-0.010,+0.018]  2W/3L
p=1.000  clears0=False
  drop duration                             ?F1(full-drop)=-0.001  CI[-0.004,+0.000]  0W/2L
p=0.500  clears0=False

=== Incremental add-onto-shuttle (B - A) ===
  shuttle->+person              F1 0.307->0.286  ?F1=-0.021  CI[-0.165,+0.162]  2W/4L p=0.688
clears0=False
  shuttle->+audio               F1 0.307->0.375  ?F1=+0.068  CI[-0.002,+0.140]  4W/2L p=0.688
clears0=False
  shuttle+person->+audio        F1 0.286->0.366  ?F1=+0.079  CI[-0.054,+0.203]  4W/2L p=0.688
clears0=False
  shuttle->+person+audio        F1 0.307->0.366  ?F1=+0.059  CI[-0.039,+0.156]  4W/2L p=0.688
clears0=False

=== Gate-A ON vs OFF (static CONTROL vs net_crossings>=2) ===
  CONTROL F1=0.388 (P=0.313 R=0.539)  segratio=1.68  n_pred=431
  Gate-A  F1=0.462 (P=0.477 R=0.468)  segratio=1.01  n_pred=235
  ?F1=+0.074 CI[+0.042,+0.105] 6W/0L p=0.031 clears0=True

=== Gate-B ON vs OFF (static CONTROL vs players_in_court>=N) ===
  N>=1: CONTROL F1=0.388 -> Gate-B F1=0.388  ?F1=+0.000 CI[+0.000,+0.000] 0W/0L p=1.000
clears0=False n_pred 431->431
  N>=2: CONTROL F1=0.388 -> Gate-B F1=0.389  ?F1=+0.000 CI[+0.000,+0.001] 1W/0L p=1.000
clears0=False n_pred 431->430
```

```
  N>=3: CONTROL F1=0.388 -> Gate-B F1=0.307  ?F1=-0.081 CI[-0.244,+0.000] 0W/1L p=1.000
clears0=False n_pred 431->274
  N>=4: CONTROL F1=0.388 -> Gate-B F1=0.283  ?F1=-0.105 CI[-0.305,+0.002] 1W/2L p=1.000
clears0=False n_pred 431->231

=== Gate-A + Gate-B stacked vs Gate-A alone ===
  GateA + B(N>=1) vs GateA: F1 0.462->0.462 ?F1=+0.000 CI[+0.000,+0.000] 0W/0L p=1.000
  GateA + B(N>=2) vs GateA: F1 0.462->0.462 ?F1=+0.000 CI[+0.000,+0.000] 0W/0L p=1.000

Wrote scratch\gate_ab_ablation_result.json
```

33. assistant (2026-06-14T09:52:07.651Z)

The ablation ran cleanly with real measured numbers. Key findings:

- **Gate-A ON vs OFF (static, the foundational decision)**: ?F1 **+0.074**, CI **[+0.042, +0.105]** clears 0, **6W/0L, p=0.031** ? reproduced exactly. Over-seg ratio **1.68?1.01**, n_pred 431?235.
- **Gate-B**: at the production-relevant counts it's a no-op (N?1/N?2: nearly every window already has ?1-2 detected persons because no court polygons ? spectators counted) and at N?3/4 it HURTS (drops real rallies). Confirms the "counts spectators" caveat. Gate-B adds nothing on top of Gate-A.
- **Leave-one-out (learned columns)**: only **audio** carries positive marginal info (+0.079, but CI spans 0); person ? neutral/slightly negative; net_crossings-as-a-column ? neutral (its power lives in the GATE, not the column); velocity/duration ? zero marginal.

One thing to verify: the Gate-A static A/B uses the same recall-oriented CONTROL proposal set as the archive. Let me confirm the CONTROL F1 (0.388) matches the archive and that the gate's `merge_gap` default matches. It does (0.388 / 0.462 / segratio 1.68?1.01 all match the archive exactly).

Now let me also double-check one subtlety: the leave-one-out "full" learned F1 is 0.366, matching the P4 endpoint. Good ? internally consistent. Let me verify the proposal-recall ceiling numbers per video and pull the per-video Gate-A deltas for the table.

34. user (2026-06-14T09:52:13.364Z)

```
Exit code 1
python.exe: ScriptBlock should only be specified as a value of the Command
parameter.
```

35. assistant (2026-06-14T09:52:16.596Z)

The multi-line `python -c` with the for-loop f-strings tripped PowerShell parsing. Let me write it to a small scratch file instead.

36. assistant (2026-06-14T09:52:21.104Z)

```
import json
d = json.load(open('scratch/gate_ab_ablation_result.json', encoding='utf-8'))
ga = d['gate_a']
print('Gate-A per-video DeltaF1 (B-A):')
for k, v in ga['per_video_dF1'].items():
    print(f'  {k:<24s} {v:+.3f}')
print()
print('CONTROL F1@k:', {k: round(v, 3) for k, v in ga['f1k_a'].items()})
print('Gate-A  F1@k:', {k: round(v, 3) for k, v in ga['f1k_b'].items()})
print()
print('Audio leave-out per-video (full-13 vs drop-audio):')
for r in d['leave_one_out']:
    if r['signal'] == 'audio':
        for vid in r['per_video_full']:
            print(f"  {vid:<24s} full={r['per_video_full'][vid]:.3f} drop-
audio={r['per_video_dropped'][vid]:.3f}")
```

```
print()
print('Person leave-out per-video (full-13 vs drop-person):')
for r in d['leave_one_out']:
    if r['signal'] == 'person':
        for vid in r['per_video_full']:
            print(f"  {vid:<24s} full={r['per_video_full'][vid]:.3f} drop-
person={r['per_video_dropped'][vid]:.3f}")
```

37. user (2026-06-14T09:52:21.118Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\scratch\gate_ab_detail.py (file state is current in your context ? no need to Read it
back)
```

38. user (2026-06-14T09:52:27.634Z)

```
Gate-A per-video DeltaF1 (B-A):
  adarsh_avi_singles       +0.125
  gbaaddy                  +0.043
  kushagra_singles         +0.009
  largetest_doubles        +0.074
  mahadevpura_singles      +0.117
  testlarge_short          +0.075

CONTROL F1@k: {'f1@10': 0.645, 'f1@25': 0.603, 'f1@50': 0.388, 'segment_count_ratio': 1.675}
Gate-A  F1@k: {'f1@10': 0.748, 'f1@25': 0.698, 'f1@50': 0.462, 'segment_count_ratio': 1.014}

Audio leave-out per-video (full-13 vs drop-audio):
  adarsh_avi_singles       full=0.677 drop-audio=0.645
  gbaaddy                  full=0.102 drop-audio=0.000
  kushagra_singles         full=0.105 drop-audio=0.128
  largetest_doubles        full=0.596 drop-audio=0.324
  mahadevpura_singles      full=0.492 drop-audio=0.222
  testlarge_short          full=0.222 drop-audio=0.400

Person leave-out per-video (full-13 vs drop-person):
  adarsh_avi_singles       full=0.677 drop-person=0.568
  gbaaddy                  full=0.102 drop-person=0.250
  kushagra_singles         full=0.105 drop-person=0.150
  largetest_doubles        full=0.596 drop-person=0.614
  mahadevpura_singles      full=0.492 drop-person=0.525
  testlarge_short          full=0.222 drop-person=0.143
```

39. assistant (2026-06-14T09:52:39.813Z)

```
All numbers confirmed and internally consistent. Per-video Gate-A is positive on all 6 (6W/0L
confirmed at the per-video level). Now I have everything for the comprehensive table. Let me
assess the Gemini question before writing the doc.

**Gemini decision reasoning**: The persisted-feature ablation answers the fusion-layer question
(which signals carry marginal information, and whether Gate-A as a decision mask is a real win)
decisively and at zero cost. A served end-to-end Gemini run would measure something *different*
? final highlight-reel quality with Gate-A flipped on in the live served path. But:
1. The decision under consideration is the **Gate-A default flip in the fusion/proposal layer**,
which is upstream of Gemini and already measured against golden labels (6/6, p=0.031).
2. There are **no golden labels** for any new served-pipeline footage, so a Gemini run produces
divergence (a_only/b_only), not F1 ? it can't move a numbers-driven decision, only illustrate
it.
3. The known GX010125 offline re-score (45?27 rallies, 18 low-crossing windows dropped)
**already corroborates** the served-side behavior at zero Gemini cost.

So a served Gemini run is **not warranted** to make this decision. I'll flag it with expected
cost/time for the owner as an optional confidence-builder. Let me write the doc.
```

40. assistant (2026-06-14T09:54:16.758Z)

Gate-A/B foundational decision ? golden-set numbers + implications

Owner's framing: *NUMBERS INFORM, FOUNDATIONS DECIDE.* Extensibility, scope of exploration, and rigid foundational choices weigh more than raw ?F1 ? but this must be a **data-INFLUENCED** choice, so every number below is **actually measured** (no fabrication) on the n=6 owned golden corpus, IoU ? 0.5, with **honest stats (C1: bootstrap 95% CI + paired sign test; never a bare mean at n=6)**.

Repro (CPU, seconds, no GPU/Gemini):
`PYTHONPATH=<repo> python scratch/gate_ab_ablation.py` ? `scratch/gate_ab_ablation_result.json`.
Reuses ONLY the shipped harness (`backend/eval/experiment.compare` + `backend/eval/eval_stats`) over
the 6 persisted `output/*_golden_features.json`. All 6 JSONs already carry the audio columns ? no
`--augment` step was needed.

Corpus (the 6 labelled matches in `output/human_lovo_manifest.json`):

video	candidates	golden rallies	proposal-recall ceiling	note
adarsh_avi_singles	172	96	0.833	clean singles
largetest_doubles	86	49	0.816	doubles
mahadevpura_singles	57	31	0.645	single-court-ish
gbaaddy	82	48	**0.312**	multi-court
testlarge_short	14	6	0.500	short
kushagra_singles	20	32	**0.125**	multi-court

Multi-court matches are **windowing-capped** (the decision layer cannot recover rallies the proposal
windowing missed) ? their low absolute F1 is a proposal-recall problem, not a gate/fusion failure.

A. The two mechanisms (this is the foundational distinction)

The golden JSONs hold **two structurally different things**, and the Gate-A/B decision lives in the
first one:

- **GATES = decision masks** over the recall-oriented CONTROL proposal set. `Gate-A = net_crossings ? 2`
  and `Gate-B = players_in_court ? N` are hard keep/drop rules applied *before* (or instead of) any
  learned scorer. In the harness these are `Variant.min_crossings` / `Variant.min_players`
  (`_gate_mask` in `experiment.py`). **This is what the foundational decision is about.**
- **FEATURE COLUMNS = learned-fusion inputs.** shuttle (5: `duration, peak_velocity, mean_velocity,
  active_ratio, net_crossings`) + person (5) + audio (3) = **13 columns** a LOVO LogisticRegression
  weights. Here a signal contributes "softly" ? the scorer decides how much to trust it. Removing one
  group and re-measuring held-out ?F1 = that group's **marginal contribution**.

Note `net_crossings` appears in BOTH: as a **gate** (hard `?2`, the Gate-A decision) and as a **soft
feature column**. They are not the same lever ? and the numbers below show the *gate* form is where
all the power is.

B. Comprehensive numbers table

B1. Gate-A ON vs OFF ? the foundational decision (static, CONTROL vs `net_crossings ? 2`)

variant	F1	P	R	seg-count ratio	# predicted segs
CONTROL (merge-all, no gate)	0.388	0.313	0.539	1.68	431
Gate-A (`min_crossings ? 2`)	**0.462**	**0.477**	0.468	**1.01**	235

?F1 = +0.074, 95% CI **[+0.042, +0.105]** (clears 0), sign test **6W / 0L, p = 0.031**. Per-video ?F1 (all positive): adarsh +0.125, mahadevpura +0.117, testlarge +0.075, largetest +0.074, gbaaddy +0.043, kushagra +0.009. **F1@k** all rise (F1@10 0.645?0.748, F1@25 0.603?0.698, F1@50 0.388?0.462). The over-seg ratio **1.68 ? 1.01** is the headline structural effect: Gate-A nearly eliminates the over-segmentation tIoU barely reflected, by pruning **196 of 431** low-crossing proposal windows (held-shuttle / single-feed / dead-time FPs ? the owner's bug). *Exactly reproduces the first-A/B archive (`docs/archives/quality-iterations/2026-06-first-ab-experiment/`).*

B2. Gate-B ON vs OFF (static, CONTROL vs `players_in_court ? N`)

Gate-B threshold	F1	?F1 vs CONTROL	95% CI	sign test	clears 0	# segs 431?
N ? 1	0.388	+0.000	[0, 0]	0W/0L	No	431 (no-op)
N ? 2	0.389	+0.000	[0, +0.001]	1W/0L	No	430 (?no-op)
N ? 3	0.307	**?0.081**	[?0.244, 0]	0W/1L	No	274
N ? 4	0.283	**?0.105**	[?0.305, +0.002]	1W/2L	No	231

Gate-B is inert-to-harmful on this corpus. At production-plausible thresholds (N?1/2) it is a **no-op** ? nearly every window already shows ?1?2 detected persons because there are **no court polygons**, so `players_in_court` counts **spectators + adjacent-court players**, not just the two on the active court. Pushing N higher to force discrimination starts **deleting real rallies** (N?3: ?0.081; N?4: ?0.105). **Stacked on Gate-A, Gate-B adds exactly 0.000** (N?1 and N?2 both leave Gate-A's 0.462 unchanged). Gate-B is **not measurable as a win until court polygons exist.**

B3. Leave-one-signal-out ? marginal contribution to the learned 13-column fusion (LOVO)

(delta = F1(full 13 cols) ? F1(full minus that signal); positive ? the signal helps the scorer)

Full learned LOVO reference: **F1 = 0.366** (P 0.453 / R 0.341).

signal dropped	F1 full	F1 dropped	marginal ?F1	95% CI	sign test	clears 0
audio (3 cols)	0.366	0.286	**+0.079**	[?0.054, +0.203]	4W/2L (p=0.688)	No
`net_crossings` (as a soft column)	0.366	0.382	**?0.014**	[?0.083, +0.027]	4W/1L (p=0.375)	No
person (5 cols)	0.366	0.376	**?0.009**	[?0.074, +0.055]	2W/4L (p=0.688)	No
velocity block (peak+mean+active)	0.366	0.364	+0.002	[?0.010, +0.018]	2W/3L (p=1.000)	No
`duration`	0.366	0.367	?0.001	[?0.004, 0]	0W/2L (p=0.500)	No

Reading: the only signal carrying *positive* marginal information for the learned scorer is **audio** (+0.079; the scorer loses the most when audio is removed). **`net_crossings` as a soft column is ~neutral-to-slightly-negative (?0.014)** ? its discriminative power is fully captured by the **hard Gate-A** form, not the learned column. Person is mildly negative (the no-court-polygon noise). Velocity/duration are ~zero marginal. **Every CI spans 0** (n=6) ? see the caveat.

B4. Incremental add-onto-shuttle (the P3/P4 framing, for cross-reference)

step	F1 A?B	?F1	95% CI	sign test	clears 0
shuttle ? +person (P3)	0.307 ? 0.286	**?0.021**	[?0.165, +0.162]	2W/4L (p=0.688)	No
shuttle ? +audio	0.307 ? 0.375	**+0.068**	[?0.002, +0.140]	4W/2L (p=0.688)	No
shuttle+person ? +audio (P4)	0.286 ? 0.366	**+0.079**	[?0.054, +0.203]	4W/2L (p=0.688)	No
shuttle ? +person+audio	0.307 ? 0.366	+0.059	[?0.039, +0.156]	4W/2L (p=0.688)	No

Consistent with the leaderboard (P3 ?0.021, P4 incr. +0.079). Audio is the lead learned cue; person is a wash on its own. Note **audio added directly to shuttle (+0.068, CI [?0.002, +0.140])** is the cleanest audio read (drops the person handicap) and its CI **barely** touches 0 ? the most promising near-promotable signal.

B5. Existing baselines (context ? IoU 0.5 vs human golden, from `eval_baselines/` + QUALITY_ITERATIONS)

path	Mahadevpura (clean-ish)	Kushagra (multi-court)	corpus-mean F1
Heuristic (velocity windowing, LOVO n=6)	0.657	0.157	0.48
Gen-0 owned head (per-frame proto, LOVO n=6)	0.744	0.561	0.583
Two-stage WASB+Gemini refine	0.83	?	?
Gemini wrapper (`gemini-flash-latest`)	**0.93**	**0.66**	?

Wrapper baseline cost note: **well under $1 for two videos** at flash pricing (serial, ~14 chunk requests). Multi-court drops the commodity wrapper 0.93 ? 0.66 ? that 0.66 is the owned model's ship target. **Caveat on cross-comparing rows:** the fusion/Gate numbers (§B1?B4) are scored on a *recall-oriented proposal* candidate set (CONTROL ? 0.388 mean), a different operating regime than the heuristic/Gen-0/wrapper rows; don't read 0.462 vs 0.93 as "Gate-A loses to Gemini" ? they answer different questions (proposal-decision lift vs end-to-end detector quality).

C. Gate-A default flip ? implications (numbers AND foundations)

C1. What the numbers say
Gate-A is the **only** lever in this whole study that clears the honest promotion bar: ?F1 +0.074 with a CI that excludes 0 **and** a 6/0 sign test (p=0.031) ? it improves F1 on **every** golden match and slashes over-segmentation (1.68?1.01). It is cheap, GPU-free, deterministic, and was already corroborated on real footage (GX010125 offline re-score: 45 ? 27 rallies, 18 low-crossing FP windows dropped). By the repo's own "promote only on measured lift" rule, **Gate-A qualifies and Gate-B does not.**

C2. Extensibility ? does defaulting Gate-A ON constrain or enable future signal fusion?

It ENABLES, and here's the structural reason. Gate-A is a **proposal-stage precision filter**, not a verdict. It prunes structurally-impossible-to-be-a-rally windows (a held shuttle never crosses the net twice) *before* the learned scorer / Gemini ever sees them. That is strictly **complementary** to adding more signals:

- It **raises the floor every later cue builds on.** The full-13 learned fusion is measured on the *un-gated* CONTROL set (F1 0.366); a learned fusion measured **on top of a Gate-A-pruned set** starts from 0.462, with the worst FP class already removed ? every cue's job gets easier, not harder.

- It does **not** consume or mask any feature column. The persisted superset (shuttle+person+audio, 13 cols) stays intact in the JSONs; Gate-A only changes which *windows survive to scoring*. So audio (the promising +0.079 lead) and any future cue are added exactly as before ? Gate-A is orthogonal to the fusion layer, living one stage upstream.
- The harness already treats it as a first-class, swappable `Variant` knob (`min_crossings`), so a future cue that *subsumes* Gate-A's signal can be A/B'd against it and Gate-A flipped back off via config ? **the experiment seam is preserved.**

The one real constraint to name honestly: Gate-A is a **hard gate**, not a soft feature. The leave-one-out numbers (§B3) show the *soft* `net_crossings` column is ~neutral (?0.014) while the *hard* `?2` gate is the win ? i.e. the value is in the thresholding, which a learned scorer did **not** reproduce from the column at n=6. Defaulting the hard gate ON therefore bakes in a **rule-based pre-filter alongside the learned path**. That is a deliberate two-stage shape (rule-gate ? learned/Gemini scorer), not pure end-to-end learning. It is reversible (config flag) and standard (proposal-filtering before scoring is textbook Temporal Action Localization), but it IS a foundational commitment to a **hybrid rule+learned pipeline**, which the owner should make eyes-open.

C3. Scope of exploration ? what does the default flip open vs close?

- **Opens:** a clean, higher-precision substrate for the *next* signal hunt (audio first). Every future cue gets measured against the Gate-A floor, so a real lift is more likely to be a *real* lift and less likely to be "it cleaned up held-shuttle FPs the gate would have caught anyway."
- **Opens:** the held-shuttle / single-feed / dead-time FP class is closed structurally, freeing the exploration budget to chase the **harder, temporal multi-court confound** (the documented dead end for spatial/loudness heuristics ? only the learned temporal model cracks it).
- **Closes nothing important:** because the persisted superset is untouched, no future ablation is foreclosed. The only thing it "closes" is re-litigating held-shuttle FPs by other means.

C4. Foundational structure ? is it hard to reverse?

No ? and this is the key de-risking fact. Gate-A as shipped is `indexing.fusion.min_crossings` (default 0 = OFF today). Flipping the default to 2 is a one-line config change; rollback is flipping it back (the repo's stated discipline: "rollback = flip a config flag, never `git revert`"). It does not alter the DB schema, the persisted feature superset, or any registry contract. The *served-side* flip is the only part still owner-gated, and only because the production fusion segmenter (P2) is unverified on a live real-video run ? **that's a verification gap, not a reversibility problem.**

The Gate-B side IS a foundational question, but the data says **don't build on it yet**: it is inert without court polygons and harmful when forced. Committing to Gate-B now would bake a **dependency on a court-mask source** (manual polygon vs auto-homography ? an open owner decision) into the default path for **zero measured benefit**. Keep Gate-B default-OFF; revisit only after court polygons exist, then re-measure person *and* Gate-B together (the current ?0.021 person result is partly a `court_polygon=None` artifact).

D. Recommendation framing (numbers + foundations, not numbers alone)

1. **Promote Gate-A to the default on the fusion/proposal path** (`indexing.fusion.min_crossings = 2`).
   - *Numbers:* the only lever that clears the honest bar (+0.074, CI clears 0, 6/0, p=0.031; over-seg 1.68?1.01) ? measured, reproduced, and corroborated on real footage.
   - *Foundations:* it's an upstream, orthogonal, reversible precision pre-filter that *raises the floor* for every future cue without consuming any feature or foreclosing any ablation. The only eyes-open commitment is to a **hybrid rule+learned two-stage shape**, which is standard and config-reversible.
   - *Gate on:* the live served-side flip stays owner-gated **only** until one real fusion-path run confirms P2 behaves as the offline re-score predicts (the GX010125 evidence already points the right way). This is verification, not a numbers question.

2. **Keep Gate-B default-OFF.** No measured benefit (no-op at N?2, harmful at N?3, +0.000 stacked on Gate-A). It is **blocked on court polygons**, not on more data. Do not bake a court-mask dependency into the default path for zero lift. Re-open Gate-B *and* the person cue together once polygons exist.

3. **Make audio the next promotion candidate, not person.** Audio is the strongest learned cue (leave-out +0.079; add-onto-shuttle +0.068 with a CI that only just touches 0). The honest blocker is n=6: every fusion CI spans 0. **Growing the golden corpus (Roora) is the highest-ROI unblock** ? it is the only thing that can turn the +0.07-ish audio lead and the person re-measurement into confirmable results.

4. **Be honest about the central caveat:** *at n=6 every learned-fusion CI spans 0.* Gate-A is the lone exception (its CI clears 0 and the sign test is significant) precisely because it's a strong, simple, structural rule that wins on all 6 videos ? not a marginal learned weight. So the numbers **decisively** support Gate-A and **decisively reject Gate-B**, but only **suggest** (cannot confirm) audio and **cannot exonerate** person until N grows. This is a data-influenced decision where the data is strong exactly where the foundational choice is being made (Gate-A) and weak where it isn't (the soft cues).

E. Is a served end-to-end Gemini run on the golden videos warranted? ? **NO (for the decision).**

Reason. The Gate-A/B decision lives in the **proposal/fusion layer, upstream of Gemini**, and is already settled by the persisted-feature ablation against golden labels (6/6, p=0.031, CI clears 0). A served Gemini run would measure a *different* thing ? final highlight-reel quality with the gate flipped on in the live path ? and it **cannot move a numbers-driven decision** because:

- There are **no golden labels** for new served-pipeline footage, so the output is divergence (`a_only`/`b_only` windows), not F1 ? illustrative, not decisive.
- The golden videos' served behavior under Gate-A is **already corroborated** by the GX010125 offline re-score (45 ? 27 rallies, 18 low-crossing FP windows dropped, ~1.7 min), at zero Gemini cost.
- The remaining owner-gate on the served-side flip is a **verification** that the production fusion segmenter (P2) behaves as the offline path predicts ? a cheap single real-video run, **not** a golden-corpus Gemini sweep.

What a Gemini run WOULD be worth (optional, owner's call ? flagged, not run): a *confidence-builder / demo*, not a decision input ? run the served fusion pipeline on ONE golden match with `min_crossings` OFF vs 2 and eyeball the two reels to confirm P2 drops the held-shuttle windows end-to-end.
- **Expected cost/time:** ~1 golden match ? tens of minutes wall-clock (GPU WASB/person pass + ffmpeg + serial Gemini chunks); Gemini spend **< $1** at flash pricing (the wrapper baseline did two videos for under $1). Two reels (gate OFF/ON) ? **2× that**, still sub-$2.
- **Recommendation:** worth it once, as the **owner-gated served-side verification** of P2 (item D1's gate), but **not** as input to the Gate-A/B foundational decision ? the CPU ablation already decided that. Do not burn a multi-video Gemini sweep; it adds cost, not decision value.

Artifacts

- Runner: `scratch/gate_ab_ablation.py` · detail: `scratch/gate_ab_detail.py`
- Raw measured result: `scratch/gate_ab_ablation_result.json`
- Reuses: `backend/eval/experiment.py`, `backend/eval/eval_stats.py`, `backend/eval/segmentation_metrics.py`, `output/*_golden_features.json` (the persisted substrate).
- Cross-refs: `docs/QUALITY_ITERATIONS.md`, `eval_baselines/experiment_leaderboard.json`, `docs/archives/quality-iterations/2026-06-first-ab-experiment/README.md`.

41. user (2026-06-14T09:54:16.780Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\GATE_AB_NUMBERS_AND_IMPLICATIONS.md (file state is current in your context ? no need to Read it back)

42. assistant (2026-06-14T09:54:20.218Z)

Let me do a final consistency check ? verify the doc's headline numbers against the raw JSON, and confirm the n_pred pruning count (431?235=196) is right.

43. user (2026-06-14T09:54:24.330Z)

GateA dF1 0.074 CI 0.042 0.105 6 W 0 L p 0.031
npred 431 -> 235 pruned 196
full learned F1 0.366

44. assistant (2026-06-14T09:54:47.672Z)

All headline numbers confirmed (Gate-A ?F1 +0.074, CI [+0.042, +0.105], 6W/0L p=0.031, 196 windows pruned, full learned F1 0.366). The doc is consistent with the measured data. Everything was written only to `scratch/`, tracked code was read-only, no `--augment` was needed (JSONs already had audio), and no Gemini/GPU was burned.

Report

I collected all golden-set numbers for the Gate-A/B foundational decision, ran a leave-one-signal-out ablation on the 6 persisted feature JSONs, and wrote the implications doc. All numbers are actually measured (CPU, seconds, reusing the shipped harness + honest stats). **Audio columns were already present in all 6 JSONs, so `--augment` was not needed.**

Deliverable: **`C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\GATE_AB_NUMBERS_AND_IMPLICATIONS.md`** (runner: `scratch/gate_ab_ablation.py`; raw: `scratch/gate_ab_ablation_result.json`).

Comprehensive numbers table (n=6, IoU?0.5, honest C1)

Gate-A ON vs OFF (the foundational decision ? static):

variant	F1	P	R	seg-ratio	#segs
CONTROL	0.388	0.313	0.539	1.68	431
Gate-A (?2)	**0.462**	0.477	0.468	**1.01**	235

?F1 **+0.074**, CI **[+0.042, +0.105]** (clears 0), **6W/0L, p=0.031**; 196 windows pruned; positive on all 6 videos. Exactly reproduces the first-A/B archive.

Gate-B (players_in_court?N): no-op at N?2, **harmful** at N?3 (?0.081) / N?4 (?0.105); **+0.000 stacked on Gate-A**. Inert without court polygons (counts spectators).

Leave-one-signal-out marginal ?F1 (full 13-col learned LOVO F1=0.366; delta = full ? dropped): audio **+0.079**, net_crossings-as-column **?0.014**, person **?0.009**, velocity +0.002, duration ?0.001 ? **every CI spans 0**.

Incremental: +person (P3) ?0.021; +audio +0.068 (CI [?0.002,+0.140]); +audio on person (P4) +0.079.

Existing baselines: heuristic 0.48 mean / Gemini wrapper 0.93 clean?0.66 multi-court / Gen-0 0.583.

Key implications

- **Gate-A is the only lever clearing the honest bar.** It's an **upstream, orthogonal, reversible** precision pre-filter (one-line `indexing.fusion.min_crossings=2`, default-OFF today; rollback = flip the flag). It does **not** consume any feature column or foreclose any ablation ? the persisted 13-col superset stays intact ? so it **raises the floor** every future cue builds on. The one eyes-open commitment: the *hard gate* (not the soft `net_crossings` column, which is ~neutral at ?0.014) bakes in a **hybrid rule+learned two-stage shape** ? standard and config-reversible.
- **Gate-B is a foundational trap right now:** committing to it bakes a court-mask-source dependency into the default for **zero measured benefit**. Keep default-OFF; re-open with person once court polygons exist.
- **Central caveat (honest):** at n=6 every *learned-fusion* CI spans 0. Gate-A is the lone exception precisely because it's a strong structural rule winning 6/6, not a marginal learned weight. Audio is the lead soft cue but only *suggestive*; growing the corpus (Roora) is the highest-ROI unblock.

Recommendation framing

Promote **Gate-A** (numbers decisive + foundations enabling/reversible); keep **Gate-B OFF** (no lift, blocked on polygons); make **audio** the next promotion candidate over person; be explicit that the data is strong exactly where the foundational choice is made (Gate-A) and weak where it isn't (soft cues).

Served Gemini run ? **NO** (for the decision)

The Gate-A/B decision is in the proposal/fusion layer **upstream of Gemini** and already settled against golden labels (6/6, p=0.031). A served run can't move a numbers decision (no golden labels on new footage ? divergence, not F1) and the served behavior is already corroborated by the GX010125 offline re-score (45?27 rallies, 18 FP windows dropped, zero Gemini cost). Flagged for the owner as an **optional one-match confidence-builder / P2 served-side verification** only: ~tens of minutes wall-clock, Gemini spend **<$1 per reel** (sub-$2 for an OFF/ON pair) ? worth it once as the owner-gated served verification, not as input to this decision. Do not burn a multi-video Gemini sweep.